

RMCS

思想、架构和指南

Zihan Qin

“Harry is the best hope we have. Trust him.”

August 01, 2025

Thank you for using this book ❤️,
I hope you like it 😊

目录

1. 无下位机, RMCS 与 libRMCS	5
1.1. 故事一则	5
1.2. 无下位机	6
1.3. 无下位机控制系统 RMCS	7
1.4. 通讯核心库 libRMCS	8
2. 环境配置	9
2.1. 开发环境配置	9
2.1.1. 前置要求	9
2.1.2. 获取开发镜像	9
2.1.3. 克隆并打开仓库	9
2.1.4. 配置 VSCode	9
2.1.5. 构建 RMCS	9
2.1.6. 运行 RMCS (可选)	10
2.2. 部署环境配置	10
2.2.1. 前置要求	10
2.2.2. 获取部署镜像	10
2.2.3. 启动容器	11
2.2.4. 远程连接	11
2.2.5. 同步构建产物	11
2.2.6. 重启服务	11
2.2.7. 糖	12
3. 使用 libRMCS	13
4. RMCS 思想和架构	14
4.1. 代码风格和命名约定	14
4.2. 单位和坐标约定	15
4.2.1. 类型	15
4.2.2. 单位	15
4.2.3. 手性	15
4.2.4. 坐标轴方向	16
4.2.5. 坐标和方向向量	16
4.2.6. 二维旋转	16
4.2.7. 三维旋转	16
4.3. 设计思想	17
4.3.1. 组件	17
4.3.2. 容器	17
4.4. 启动流程	17
4.5. 开发脚本手册	18
4.6. 网络配置指南	18

5. 部分算法分析	19
5.1. 两轴云台控制器	19
5.1.1. 更新 Yaw 轴滤波器	19
5.1.2. 更新控制向量	19
5.1.3. 将控制向量转到 YawLink 下	19
5.1.4. 对控制向量进行限位	20
5.1.5. 计算并更新控制误差	21
5.2. 全向底盘控制器	21
5.2.1. 麦轮底盘建模	21
5.2.2. 全向底盘建模	22
5.2.3. 观测底盘速度	22
5.2.4. 简单闭环控制	23
5.2.5. 计算电机控制力矩	23
5.2.6. 增加约束	24
5.2.7. 问题求解	26
5.3. 舵轮底盘控制器	26
5.3.1. 舵轮底盘建模	26
5.3.2. 观测底盘速度	27
5.3.3. 闭环底盘速度	27
5.3.4. 计算电机控制力矩	28
5.3.5. 增加约束	31
5.3.6. 问题求解	33
6. 通讯架构迭代	39
6.1. USB CDC(Communication Device Class, 通信设备类)	39
6.2. EtherCAT (工业以太网)	40
6.3. USB 同步/等时传输 (Isochronous)	41
6.4. USB 批量传输 (Bulk)	42
Index of Figures	44

1. 无下位机，RMCS 与 libRMCS

作为本书的序言部分，也为了回答“为什么要做无下位机”这个关键问题，笔者认为一定得花些笔墨，讲讲自身的一些经历与思考。

1.1. 故事一则

笔者在大一入队时，作为视觉组负责人研发步兵和哨兵的自瞄。在从零编写整个视觉算法的过程中，与电控联调，是笔者一向最头疼的事项。

在当时队伍的电控架构中，每辆车都拥有一份独立的、没有版本管理的代码，分别由不同的队员维护，即使两辆车的机械结构一致也是如此。而这些独立的代码间，唯二的同步方式是通过 U 盘或 QQ 群，唯一的备份方式是压缩为 zip 包。

对于视觉而言，这自然是一个令人头疼的工作环境。想象一下，当你与一位车辆的负责队员商议好通讯协议，并花费几天时间调试完成所有功能后，想着终于可以放松一下，但发现后面还有五辆车等着你联调。每辆车的代码都不一样，且没有版本管理，无法追踪修改记录。每次联调都需要花费大量时间，等待负责的电控队员修改代码，适配协议。如果电控队员修改正确，则万事大吉，反之，则需要再次等待他修改好代码，然后再次测试。如果运气不好，反复测试都不通过，你还需要不断修改视觉代码，构造特定的测试数据，辅助电控队员找到问题所在。

令笔者印象深刻的是该赛季无人机的联调过程。

在苦等负责无人机的电控同学修改代码五天后，笔者开始了第一次联调，但联调尚未开始便中道崩殂。因为无人机上电后，云台完全无法移动，无论怎么移动遥控器的拨杆，云台始终像焉白菜一样绵软无力。在三小时的研究后，电控同学表示没有头绪，决定来日再战。由于没有版本管理，笔者震惊地发现，就连让电控代码回到几天前的版本也无法做到。

两天后，电控同学表示问题已解决，可以再战。笔者高兴地下楼调车，但却发现这次的云台虽然不再绵软无力，但稍微一碰便开始颤抖，发出巨大的噪音。电控同学又调了两小时 PID 未果，只得再次休战。笔者不断追问是否可以再次尝试寻找较为古老的代码版本，于是在电控同学不懈的搜索下，终于在硬盘中的一个角落发现了旧版本的压缩包。

又经过电控同学两天的研究，总算找到了问题所在：上任学长传下的代码中，设置云台控制量的代码被夹在一大坨被注释掉的代码里。新负责的电控同学在修改代码，加入自瞄功能时，为了让代码更加整洁，删掉了一些被注释的代码，却意外把夹在注释中的核心控制代码给删了，这导致了第一次联调时云台的不举。第二次联调时，电控同学在另外的位置编写了新控制代码，但观测量由陀螺仪变为了编码器，导致原先的 PID 参数完全无法工作。

终于，在解决问题所在后的夜晚，笔者终于开始了一次真正意义上的联调。这次联调也并不顺利，在数次解决串口通讯问题后，电控同学终于成功收到了自瞄发来的目标角度增量值。但笔者还没来得及高兴，下一步的测试就表明，云台正确跟踪装甲板一段时间后，就会陷入疯转，且除非重启电控代码，否则无法使云台停下。

由于问题是偶发性的，且触发较为随机，复现较为困难，电控同学进行了两小时的调试无果。最后笔者通过反复手动构造数据，成功探索出了问题所在：当时场地灯光较暗，视觉代码出现了偶发的误识别现象，其中一种误识别的情况解算出的装甲板位置与 (0, 0, 0) 非常接近，导致弹道解算无解，出现异常值 NaN。在当时的视觉代码中，这个异常值没有作为特例被排除，而是

原模原样的发给了电控。电控也没有对这个异常值加以处理，NaN可能进入了某个变量（如PID环的积分量）中，进而导致以后所有输出均异常，云台不受控制地疯转。

在视觉代码中加入判断异常值的代码后，云台终于正常工作了。此时距离笔者提出联调需求，已经过去了整九天。

但显然，这只是联调过程中的其中一次，在联调完成一辆车后，笔者还需要重复上述过程，联调其它车辆。而这样的联调循环，同样不止一次。需求会变更，通讯协议也会变更。例如，为了增加为自瞄锁定的敌方绘制方框功能，需要在通讯协议中加入 `rect_x`、`rect_y` 等参数，让电控以这个值在UI平面上绘制方框。而“在UI平面上绘制方框”这个功能，笔者等待了一个月，才得以联调第一辆车；再例如，为了增加打符功能，需要让电控发送“操作手是否按下G键”信息，以保证自瞄可以按操作手的需求，分别锁定车和符，这又是一轮繁琐的调试。

由于不同的车辆于不同的时间进行联调，尚未完全联调完成时，必然会造成一个中间状态：不同的车辆拥有不同的协议，这进一步增加了痛苦程度：视觉代码需要同时保留不同版本协议的接口，并增加随时切换的冗余代码。

另外，在这版实现中，视觉与电控的通讯方式是通过串口这一古老方式进行的。稍高频的状态量只有视觉向电控发送的控制报文（~165 Hz），电控向视觉发送的状态报文（己方颜色等）则是相对低频的。但自瞄算法要在高速运动的云台中稳定工作，自然需要高频的陀螺仪状态量输入，这一问题是由外置的第三方陀螺仪解决的。C板虽然能提供约2000 Hz的陀螺仪信号，但这一信号显然无法直接传递到上位机。之前的诸多先例，已经让笔者不敢再对修改电控手上的代码提出任何复杂的要求，采购第三方陀螺仪，依照“代码能跑就不要动”精神成为了最优解。

至于哨兵决策导航所需要的各种裁判系统信息，如敌方血量、己方状态等，在当时更是不可想象。对于视觉而言，电控代码已经成为了完全的黑箱，而且是一碰就碎的脆弱玻璃箱。笔者不是没有尝试过自行修改电控代码，但此时已临近国赛，极度耦合、命名混乱、功能代码四散的淳朴C语言代码还是打破了笔者短时间内获取突破的不自量力的幻想。幸好当时（23赛季）的哨兵可以由云台手动操控位置，一定程度上解决了问题，毕竟有人类参与，导航算法便不需要那么多信息用于决策了。

在这一赛季，笔者花费了大量时间用于联调，而不是算法的研发。

这也因此在笔者心里埋下了一颗种子，一个“由视觉代码直接控制一切”的梦想。

这一梦想很快被证明是可行的。在那年的青工会上，广东工业大学分享了他们精妙绝伦的无下位机控制方案，这也使得“无下位机”这一尝试终于在队伍中被立项，并在逐步的实践中验证了其效果。

1.2. 无下位机

无下位机并非真正“没有下位机”，只是将高频率的底层控制算法移到上位机上执行。下位机不再执行计算，仅仅充当一个转发器，将从传感器收集的观测量向上位机转发，同时将来自上位机的控制量向执行器转发。

在无下位机控制方案中，上位机可以直接控制每个执行器，向执行器直接输出尽可能低层次的控制量。

由于上位机强大的内存空间、储存空间和算力,给更高级算法的使用提供了空间。同时,在上位机运行放宽了代码的运行效率要求和内存消耗要求,控制算法可以适当使用更多的抽象层级。

还是以哨兵为例,在RM赛场上普遍使用的上-下位机架构中,若将决策算法放在下位机,则难以得到由上位机处理的自瞄和定位信息,有限的算力也会制约复杂算法的使用;若将决策算法放在上位机,则难以得到由下位机处理的裁判系统、各传感器、电机等信息。

若要解决上述问题,无论将决策算法放在哪里,都需要在上下位机之间建立一个稳定高速的数据交换通道。但是,在决策算法代码的编写过程中,需要的信息是随编码过程不断变化,且无法预知的。因此,下位机的代码被需要不断的更改,以适应不断变化的信息需求。而每次通讯协议的更改,意味着需要花时间联调代码以解决潜在的漏洞。

同时维护两份不断变化的代码是困难的。因此我们很难不去思考一种可能性:将所有可观测的信息全部转发至上位机,这样下位机代码不再需要修改,上位机只需按需获取即可。

但是一旦向这个方向行走,我们很容易发现,拥有所有信息上位机,完全可以以更高的算力执行原先在下位机上执行的控制代码。此时只需维护一份上位机代码,而下位机代码不再需要变更,只需负责原样转发来自上位机的指令即可。这样,我们似乎得到了一种更简洁的控制形式:合并上下位机。

1.3. 无下位机控制系统 RMCS

在目前的代码实现中,使用无下位机代码的所有兵种共用同一个代码仓库,可以编译出一份二进制文件。且在不重新编译的情况下,通过切换不同配置文件,代码可以被配置为不同兵种并投入使用。单仓库的特性使得代码冗余极少,避免了原先下位机的“一人一仓库”难题。大部分算法都在不同的层级被各兵种复用,一个特性被加入后,其它兵种可以马上用上;一个bug被修复后,其它兵种也会立即生效。

不仅如此,在上位机上运行控制算法,使得使用仿真软件(如 gazebo、unity 等)进行 sim2real (模拟到现实)的开发方式成为可能。全自动哨兵机器人在调试时需要大面积的场地和与比赛场地相接近的障碍物。但场地道具造假不菲,大面积的场地也难以获得。采用仿真软件进行模拟测试,可以大幅降低算法开发成本,提高开发速度。

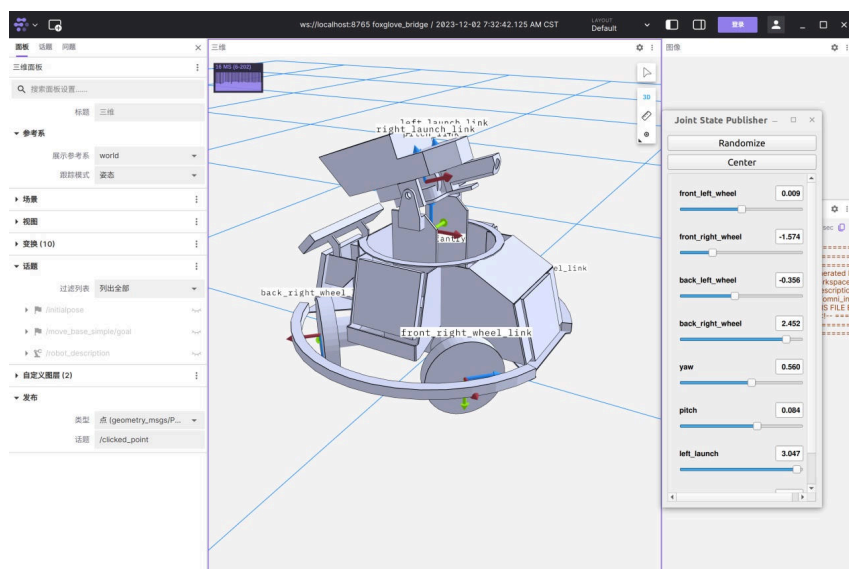


图 1 Foxglove 远程可视化界面

无下位机控制系统还可以在局域网内转发所有 `ros2 topic` (话题)、使用 `rqt`、`foxglove` 等可视化工具观察曲线，动态调整参数，这些都是传统上下位机控制框架难以做到的。

无下位机控制系统在硬件结构上也有强大的兼容性和拓展性。目前的方案采用 C 型开发板（大疆公司的出售的成品 STM32F407 开发板）作为转发板主控，用 12M USB2.0 FS 以 Bulk 协议直连上位机进行消息转发。这个方案在硬件上与上下位机方案完全相同，可以无缝的对原有车辆进行升级和替代。同时，由于上位机拥有相当数量的可拓展 USB 接口，若需要连接更多传感器或执行器设备，只需接入一块新的 C 型开发板即可。

无下位机控制系统可以解决一些上下位机控制模式下难以解决的问题。例如，在滑环线数不足的情况下，通常需要使用两块 C 型开发板，分别位于滑环上和滑环下，滑环只过一对信号线，上板间接控制底盘电机和处理裁判系统。这种控制方式复杂且容易出错，维护两份代码带来大量的时间浪费，增加需求需要同时修改两份代码。而 USB2.0 FS 信号可以过电滑环的特性，使得上下板控制从未如此方便和简洁。只需将下板 USB 的 D+、D-、和 GND 连过滑环，上位机控制电机的形式跟单板完全一致，下位机转发器代码只需一次烧录，此后无需更改。

无下位机控制系统还有丰富的第三方库可以使用。在 <https://index.ros.org/packages> 中可以浏览所有可用库，可以使用

```
sudo apt install ros-<version>-<package>
```

或 `rosdep` 简单地安装所需依赖，结合 `docker` 容器的 `build once, deploy everywhere`（一次构建，到处部署）特性，库的安装、使用和部署非常轻松。

1.4. 通讯核心库 libRMCS

TODO

2. 环境配置

2.1. 开发环境配置

2.1.1. 前置要求

- x86-64 架构

目前 RMCS 的 Docker 镜像和构建相关工具链仅考虑了 x86-64 指令集，如需要在其它指令集的平台部署 RMCS，可能需要自行构建相关镜像。如果您有对其它指令集的支持需求，可在 issue 中提出。

- 任意 Linux 发行版，或 WSL2（参见 WSL2 开发指南）

WSL2 的安装非常简单，如果没有安装 Linux 系统，我们强烈建议您尝试 WSL2。

- 安装 Docker 并配置代理（部分国家或地区）

RMCS 的开发和部署均依赖 Docker。

- 安装 Dev Containers 扩展的 VSCode

这是一个可选项，您可以使用任何支持 Docker 的编辑器，但本文仅考虑 VSCode。

2.1.2. 获取开发镜像

下载开发用 Docker 镜像：

```
docker pull qzhghi/rmcs-develop:latest
```

也可自行使用 Dockerfile 构建，参见镜像构建指南。

2.1.3. 克隆并打开仓库

克隆仓库，注意需要使用 recurse-submodules 以克隆子模块：

```
git clone --recurse-submodules https://github.com/Alliance-Algorithm/RMCS.git
```

在 VSCode 中打开仓库：

```
code ./RMCS
```

按 Ctrl+Shift+P，在弹出的菜单中选择 Dev Containers: Reopen in Container。

VSCode 将拉起一个 Docker 容器，容器中已配置好完整开发环境，之后所有工作将在容器内进行。

如果 Dev Containers 在启动时卡住很长一段时间，可以尝试这个解决方案。

2.1.4. 配置 VSCode

在 VSCode 中新建终端，输入：

```
cp .vscode/settings.default.json .vscode/settings.json
```

这会应用我们推荐的 VSCode 配置文件，你也可以按需自行修改配置文件。

在拓展列表中，可以看到我们推荐使用的拓展正在安装，你也可以按需自行删减拓展。

2.1.5. 构建 RMCS

在 VSCode 终端中输入：

```
build-rmcs
```

将会运行 `.script/build-rmcs` 脚本，在路径 `rmcs_ws` 下开始构建代码。

构建完毕后，基于 `clangd` 的 C++ 代码提示将可用。此时可以正常编写代码。

Note: 用于开发的所有脚本均位于 `.script` 中，参见 开发脚本手册(TODO)。

2.1.6. 运行 RMCS (可选)

编写代码并编译完成后，可以在开发容器中运行代码。

2.1.6.1. 确认设备接入

可以使用 `lsusb` 确定 下位机 是否已接入，若已接入，则 `lsusb` 输出类似：

```
Bus 001 Device 004: ID a11c:75f3 Alliance RoboMaster Team. RMCS Slave v2.1.2
```

在 WSL2 下，需要使用 `usbipd` 对设备进行转接。

2.1.6.2. 确认权限正确

在主机（不要在 docker 容器）的终端中输入：

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="a11c", MODE="0666"' | sudo tee /etc/udev/
rules.d/95-rmcs-slave.rules &&
sudo udevadm control --reload-rules &&
sudo udevadm trigger
```

以允许非 root 用户操作 RMCS 下位机，此指令只需执行一次。

2.1.6.3. 设置机器人名称

RMCS 启动时，需要读取机器人名称对应的 yaml 配置文件作为启动配置。使用命令：

```
set-robot <robot-name>
```

以配置机器人名称（如 `steering-infantry`）。通常此指令只需执行一次。

2.1.6.4. Launch It

使用命令：

```
launch-rmcs
```

拉起 RMCS 进程。

2.2. 部署环境配置

部署环境指部署在车上 MiniPC 的环境。若您正在学习 RMCS，这一章可以跳过。

2.2.1. 前置要求

- x86-64 架构
- 任意 Linux 发行版
- 安装 Docker

2.2.2. 获取部署镜像

下载部署用 Docker 镜像：

```
docker pull qzhghi/rmcs-develop:latest
```

如果不方便在 MiniPC 上配置代理，可以在开发机上下载镜像后，使用

```
docker save qzhghi/rmcs-runtime:latest -o rmcs-runtime.tar
```

然后使用任意方式（如 scp）将 rmcs-runtime.tar 传送到 MiniPC 上，并在 MiniPC 上执行：

```
docker load -i rmcs-runtime.tar
```

即获取部署镜像。

2.2.3. 启动容器

在 MiniPC 终端中输入：

```
docker run -d --restart=always --privileged --network=host \
-v /dev:/dev qzhghi/rmcs-runtime:latest
```

即可启动部署镜像，此后镜像将保持开机自启。

2.2.4. 远程连接

在开发容器终端中输入：

```
set-remote <remote-host>
```

其中，remote-host 可以为 MiniPC 的：

1. IPv4 / IPv6 地址 (e.g., 169.254.233.233)
2. IPv4 / IPv6 Link-local 地址 (e.g., fe80::c6d0:e3ff:fed7:ed12%eth0)
3. mDNS 主机名 (e.g., my-sentry.local)

参见 网络配置指南(TODO)。

接下来在开发容器终端中继续输入：

```
ssh-remote
```

即可在开发容器中，ssh 连接到远程的部署容器。

RMCS 的所有代码更新和调试，都基于从开发容器向部署容器的 ssh 连接。

部署容器会监听 TCP:2022 端口作为 ssh-server 端口，请注意保证端口空闲。

参见 容器设计思想(TODO)。

“Tip: GUI 可以从部署容器中穿出，尝试在 ssh-remote 中打开 rviz2。”

2.2.5. 同步构建产物

新启动的部署容器内是没有代码的，需要由开发容器上传。

在开发容器中构建完成后，可以执行指令：

```
sync-remote
```

这将拉起一个同步进程，自动将开发容器中的构建产物同步到部署容器。

同步进程除非主动使用 Ctrl+C 结束，否则不会退出，其会监视所有文件变更，并实时同步到部署容器。

“Tip: 由于 build-rmcs 采用 symlink-install 方式构建，因此对于配置文件和 .py 文件，直接修改其源文件，无需编译即可触发同步。”

2.2.6. 重启服务

RMCS 在部署容器中以服务方式启动 (/etc/init.d/rmcs)。

确认构建产物同步完毕后（以出现 Nothing to do 为标志），进入 ssh-remote，输入：

```
set-robot <robot-name>
```

设置启动的机器人类型（例如 set-robot sentry）。

接下来继续输入：

```
service rmcs restart
```

如果一切正常，其将会输出：

```
Successfully stopped RMCS daemon.  
Successfully started RMCS daemon.
```

这证明 RMCS 已成功在部署容器中启动，并会随以后的每次容器启动而启动。

接下来可以使用：

```
service rmcs attach
```

查看 RMCS 的实时输出。

“需要注意的是，service rmcs attach 本质上是连接到了一个 GNU screen 会话，因此任何按键都会被忠实地转发至 RMCS。例如，当键入 Ctrl+C 时，RMCS 会接到 SIGINT，从而停止运行。

如果希望退出对实时输出的查看，可以键入 Ctrl+A，然后按 D。

如果希望向上翻页，可以键入 Ctrl+A，然后按 Esc。

更多快捷键组合参见 Screen Quick Reference。”

2.2.7. 糖

指令 ssh-remote 后可以添加参数，参数内容即为建立链接后立即执行的指令。

例如：

```
ssh-remote service rmcs restart
```

可以在开发容器端快速重启部署容器中的 RMCS。

又如：

```
ssh-remote service rmcs attach
```

可以在开发容器端快速查看部署容器中 RMCS 的实时输出。

实际上，可以使用指令：

```
attach-remote
```

作为前者的替代（其实现就是前者）。

进一步的，还可以使用：

```
attach-remote -r
```

作为 ssh-remote "service rmcs restart && service rmcs attach" 的替代。

其在重启 RMCS 守护进程后，自动连接显示实时输出。

更进一步的，指令间还可以组合，例如：

```
build-rmcs && wait-sync && attach-remote -r
```

可以触发 RMCS 构建，wait-sync 等待文件同步完成，接下来重启 RMCS 守护进程后，显示实时输出。

3. 使用 libRMCS

TODO

4. RMCS 思想和架构

4.1. 代码风格和命名约定

RMCS 的所有代码，**必须按照** [Google 开源项目风格指南](#) 进行编码和命名，**请务必仔细地全文阅读并理解**这篇指南。

特别地，在 RMCS 中，我们对指南中的部分内容进行了微小的修改：

- 宏

禁止定义宏。

可以使用第三方库中定义的宏，如 `rclepp` 定义的 `RCLCPP_INFO` 等。

- 全局变量

禁止定义全局变量。

- 枚举命名

Google 推荐了两种命名方式，其中：

不使用：

```
enum UrlTableErrors {  
    kOK = 0,  
    kErrorOutOfMemory,  
    kErrorMalformedInput,  
};
```

总是使用：

```
enum AlternateUrlTableErrors {  
    OK = 0,  
    OUT_OF_MEMORY = 1,  
    MALFORMED_INPUT = 2,  
};
```

- 常量命名

Google 要求所有具有静态存储类型的变量都应当以如下方式命名：

```
constexpr int kDaysInAWeek = 7;
```

但在 RMCS 中，我们要求将常量和变量同风格命名。

例如，对于局部变量：

```
constexpr int bullets_per_turn = 8;
```

对于类的成员变量：

```
static constexpr int bullets_per_turn_ = 8;
```

理由：

(1) `clangd` 总是会以清晰的蓝色标识出常量，因此在命名上手动区分是意义不大的。

(2) RMCS 在开发过程中，经常会出现常量转变为变量的情况。例如，原先所有步兵拨弹盘的的拨爪数均为 8，因此我们定义 `bullets_per_turn_` 为编译期常量 (`static constexpr`)¹。但一

¹至于为什么要将“目前看起来不变”的量设置为常量，这实际上是 RMCS 的核心思想之一：Always be Lazy。参见(TODO)。

段时间后，新造的步兵拨弹盘拨爪数为 12，因此 `bullets_per_turn` 又需要修改为变量，值由配置文件读取。在这个过程中，反复修改命名是空耗精力的。

4.2. 单位和坐标约定

4.2.1. 类型

优先使用 `double` 作为浮点类型。

在定义组件间的接口 (Interface) 时，优先使用 `int64_t` 作为整数类型。

4.2.2. 单位

RMCS 的所有变量，在没有另行标注（通过命名或注释）时，**必须使用国际单位制 (SI)**。

以下是一些 SI 中常用的基本单位：

物理量	单位
长度 (length)	米 (meter)
质量 (mass)	千克 (kilogram)
时间 (time)	秒 (second)
电流 (current)	安培 (ampere)

以及一些常用的派生单位：

物理量	单位
角度 (angle)	弧度 (radian)
频率 (frequency)	赫兹 (hertz)
力 (force)	牛顿 (newton)
功率 (power)	瓦特 (watt)
电压 (voltage)	伏特 (volt)
温度 (temperature)	摄氏度 (celsius)
磁感强度 (magnetism)	特斯拉 (tesla)

特别的，与时间相关的变量，应当优先使用 `<chrono>` 库提供的强类型时间单位。如：

```
std::chrono::milliseconds timeout; // 以 int64_t 表示的毫秒数
std::chrono::duration<double> other_timeout; // 以 double 表示的秒数
std::chrono::duration<int, std::micro> another_timeout; // 以 int 表示的微秒数
```

大部分传感器和执行器的通讯协议不使用 SI，在经过硬件层处理后，应当**统一**为 SI 单位制。

4.2.3. 手性

RMCS 的所有系统都应当是右手系的，这意味着它们遵循右手定则。

4.2.4. 坐标轴方向

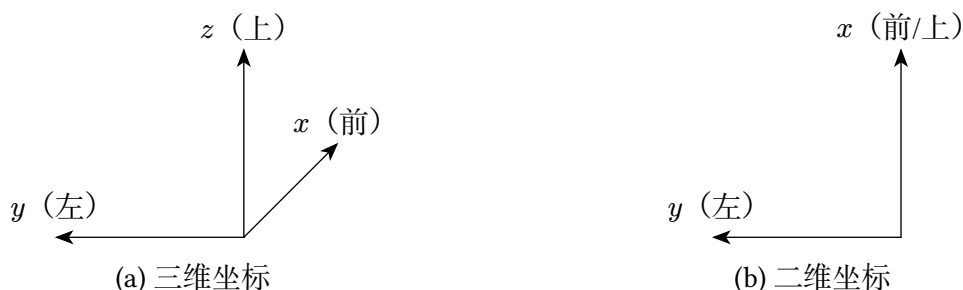


图 2 二维和三维坐标的轴方向约定（与 ROS 的约定相同）

RMCS 的所有坐标轴，在没有另行标注（通过命名或注释）时，**必须使用以上的约定**。

部分二维坐标在协议中定义了轴方向，如遥控器拨杆、鼠标移动、裁判系统 UI 等，在经过硬件层处理后，应当**统一**为以上约定的轴方向。

4.2.5. 坐标和方向向量

RMCS 中定义的所有坐标和方向向量，**除了**可根据上下文判断其坐标系的局部变量外，在没有另行标注（通过命名或注释）时，**必须使用强类型表示其坐标系**。例如：

```
rmcs_description::CameraLink::Position position;          // CameraLink坐标系下的坐标
rmcs_description::OdomImu::DirectionVector direction;    // OdomImu坐标系下的方向向量
```

这些强类型坐标本质是使用 RMCS 子模块 `fast_tf` 定义的 `Eigen::Vector3d` 和 `Eigen::Translation3d` 的包装结构。关于 `fast_tf` 和 RMCS 的坐标系树 (TF-Tree)，见(TODO)。

4.2.6. 二维旋转

RMCS 中二维旋转正方向的定义，也应当遵循右手定则。

例如，对于 RMCS 中定义的电机，其旋转的正方向**必须**与将右手拇指指向电机输出轴方向时的四指指向方向相同。

特别的，对于带减速器的电机，应当将减速器和电机看作一个**整体**，其输出轴定义为减速器的输出轴。对于带有齿轮、链轮或同步带的多级复合执行器，其输出轴应当定义为整个传动轴系**最终**的输出轴。例如：使用一个倒置的电机配合同步带驱动云台 Yaw 轴，整个 Yaw 执行器的输出轴应视为云台中轴，方向沿 z 轴向上。

4.2.7. 三维旋转

三维旋转的表示方法有很多种，RMCS 中建议的优先选用顺序及选用原因在下方列出：

1. 四元数 (Quaternion)
 - 易于运算
 - 紧凑表示
 - 无奇点
2. 轴角 (Axis-Angle)
 - 紧凑表示
 - 无奇点
3. 旋转矩阵 (Rotation Matrix)
 - 易于运算

- 无奇点
4. 分别绕 X、Y、Z 轴的滚转、俯仰、偏航值
 - 仅用于角速度
 5. Z-Y-X 欧拉角（分别绕 Z、Y、X 轴的偏航、俯仰、滚转值）
 - 欧拉角存在奇点，故通常情况下，不允许使用欧拉角

4.3. 设计思想

4.3.1. 组件

组件是 RMCS 中代码的最小单元。一个组件包含三个重要部分：**输入、输出和更新函数**。

4.3.2. 容器

4.4. 启动流程

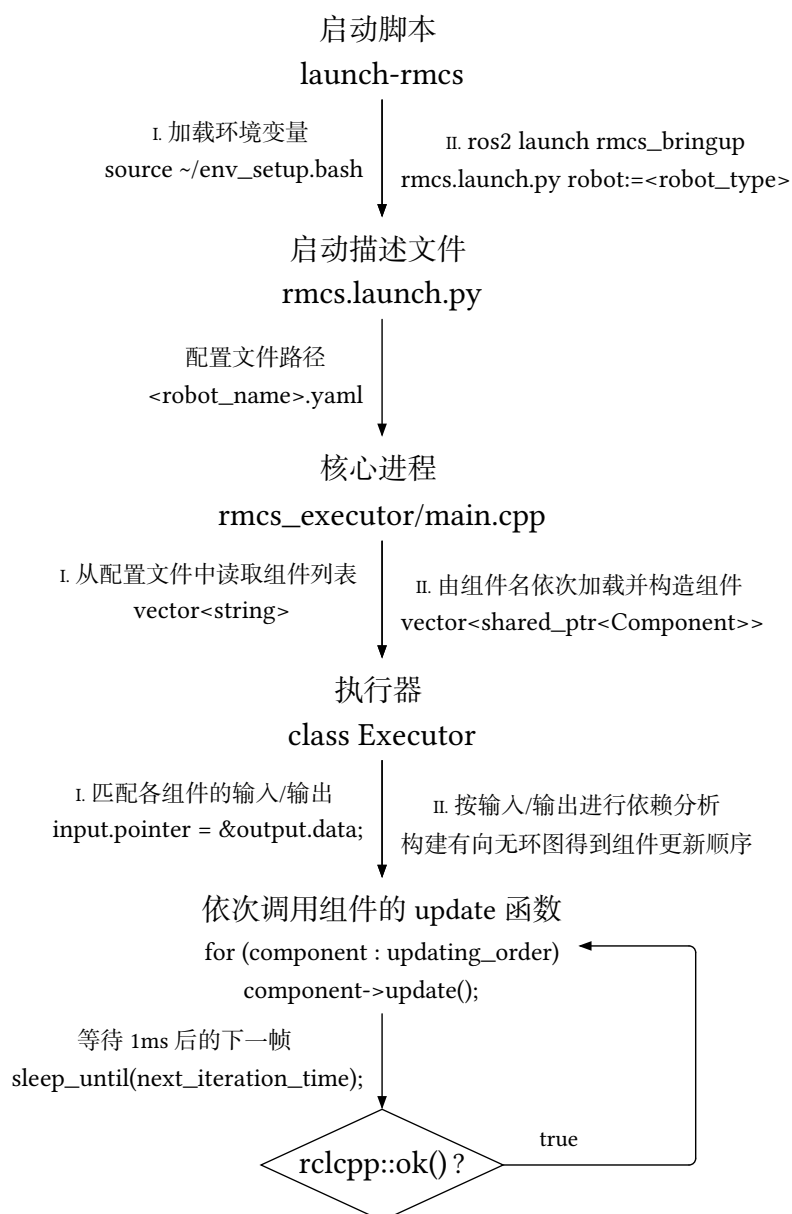


图 3 RMCS 启动流程

4.5. 开发脚本手册

TODO

4.6. 网络配置指南

TODO

5. 部分算法分析

5.1. 两轴云台控制器

云台控制器接收云台控制量（来自遥控器或鼠标的增量，或来自自瞄的目标控制向量），并输出云台的控制误差 `yaw_angle_error` 和 `pitch_angle_error`。

云台的控制频率为 1 kHz，意味着每帧计算时间 $\leq 1\text{ ms}$ 。因此云台控制器需要尽可能降低运算时间，提高实时性。

5.1.1. 更新 Yaw 轴滤波器

云台控制器储存在 OdomImu 坐标系下的 Yaw 轴向量：

```
OdomImu::DirectionVector yaw_axis_filtered_{Eigen::Vector3d::UnitZ()};
```

在每次更新开始时，控制器获取 Yaw 轴的最新位置，将其滤波后，更新该向量。

需要滤波的原因是 GM6020 电机的编码器精度太差了，若不滤波会导致剧烈的震荡。

5.1.2. 更新控制向量

云台控制器储存在 OdomImu 坐标系下的云台目标控制向量：

```
OdomImu::DirectionVector control_direction_;
```

在每次更新时，控制器可以进行下列任意一种操作：

- 使失能 `SetDisabled`
- 设置云台为水平状态 `SetToLevel`
- 设置云台目标控制向量 `SetControlDirection`
- 设置云台增量 `SetControlShift`

除第一个操作外的所有操作都可给出一个新的云台目标控制向量。

5.1.3. 将控制向量转到 YawLink 下

OdomImu 坐标系下的新控制向量可通过 Tf 树转换到 PitchLink 坐标系下。但 PitchLink 到 YawLink 的变换不能由 Tf 树（仅储存原始编码器值，容易导致震荡）进行。

考虑先由滤波后的 Yaw 轴向量计算云台 Pitch 角。将滤波后的 Yaw 轴向量通过 Tf 树转换到 PitchLink 下，设转换后的向量为 $\mathbf{q}(q_x, q_y, q_z)$ ，则云台 Pitch 角的相反数²：

$$\theta = \frac{\pi}{2} - \tan^{-1}\left(\frac{q_z}{q_x}\right)$$

显然：

$$\cos(\theta) = \frac{q_z}{\sqrt{q_x^2 + q_z^2}}, \quad \sin(\theta) = \frac{q_x}{\sqrt{q_x^2 + q_z^2}}$$

将 PitchLink 系下的向量沿 y 轴逆时针旋转 $-\theta$ ，即得到 YawLink 系下的向量。写出旋转矩阵：

$$R = \begin{pmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{pmatrix}$$

²RMCS 的角度正方向依照右手定则定义，Pitch 角（基于 y 轴）向下为正，向上为负。为防止过多符号造成混乱，在本解算中仅使用 Pitch 角的相反数（向上为正）。

于是 YawLink 坐标系下，云台的目标控制方向向量：

$$\mathbf{v} = \begin{pmatrix} x \\ y \\ z \end{pmatrix} = R\mathbf{v}_0 = \begin{pmatrix} x_0 \cos \theta - z_0 \sin \theta \\ y_0 \\ x_0 \sin \theta + z_0 \cos \theta \end{pmatrix}$$

5.1.4. 对控制向量进行限位

为简单起见，首先在平面内进行考虑。

设云台的上限位方向向量 $\mathbf{u}(a, b)$ ，其在二维平面上的角度为 α 。

设云台的目标控制方向向量为 $\mathbf{v}(x, y)$ ，其在二维平面上的角度为 β 。

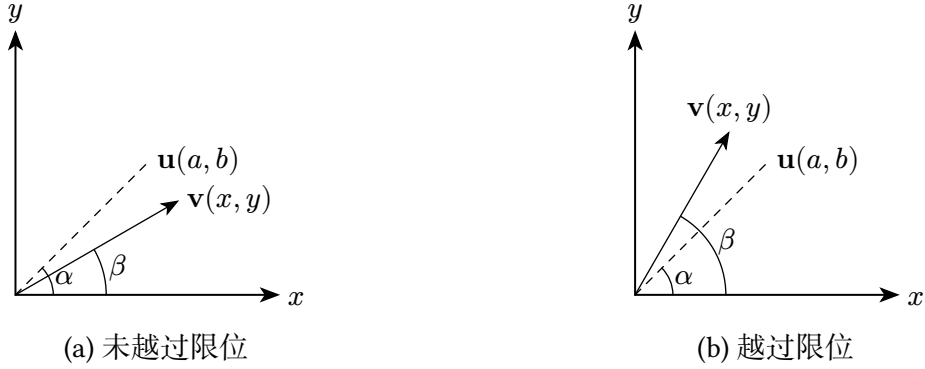


图 4 在平面内考虑云台限位

未越过限位时，目标控制向量始终在上限位方向向量下活动，有 $y \leq b$ 。

越过限位时，有 $y > b$ 。此时要使目标控制向量回到限位范围内，只需令 $\mathbf{v} = \mathbf{u}$ 即可。

接下来在三维空间中考虑云台限位，在 YawLink 坐标系下，设上限位向量 $\mathbf{u}(a, b, c)$ ，目标控制向量 $\mathbf{v}(x, y, z)$ ：

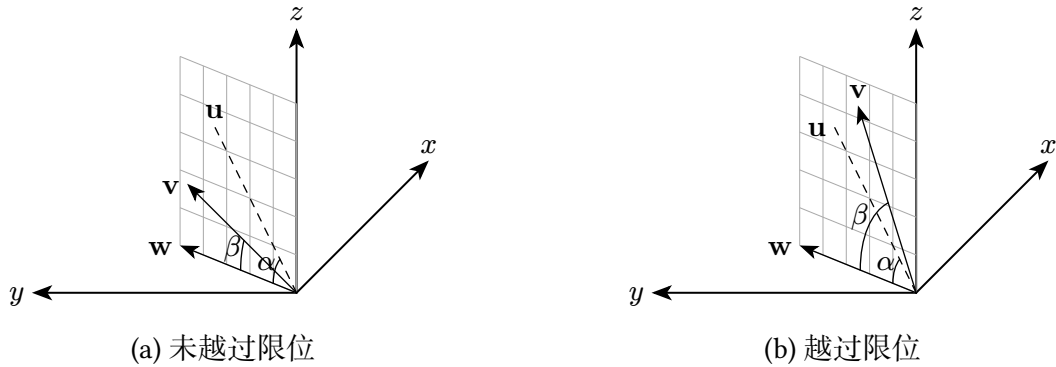


图 5 在三维空间（YawLink 系）中考虑云台限位

未越过限位时，有 $z \leq c$ 。

越过限位时，有 $z > c$ 。此时同样只需令 $\mathbf{v} = \mathbf{u}$ 即可使目标控制向量回到限位范围内。

相对于二维平面（ $\mathbf{u}$ 为定值），三维空间需要计算随 $\mathbf{v}$ 变化的 $\mathbf{u}$ 。

设 $\mathbf{w}$ 为 $\mathbf{v}$ 在 xy 平面上投影的单位向量， $\mathbf{w} = (w_x, w_y, w_z) = \text{normalized}(x, y, 0)$ ，则 $\mathbf{u} = (w_x \cos(\alpha), w_y \cos(\alpha), \sin(\alpha))$ 。

下限位的运算同理。

5.1.5. 计算并更新控制误差

如图所示，同样在 YawLink 坐标系下，设云台的目标控制方向向量为 $\mathbf{v}(x, y, z)$ ，其与自身在 xy 平面上投影的夹角为 β ；设云台的当前方向向量为 $\mathbf{n}(\cos \theta, 0, \sin \theta)$ ，其与自身在 xy 平面上投影的夹角，即云台 Pitch 角的相反数为 θ ；设两向量在 xy 平面上投影的夹角为 γ 。

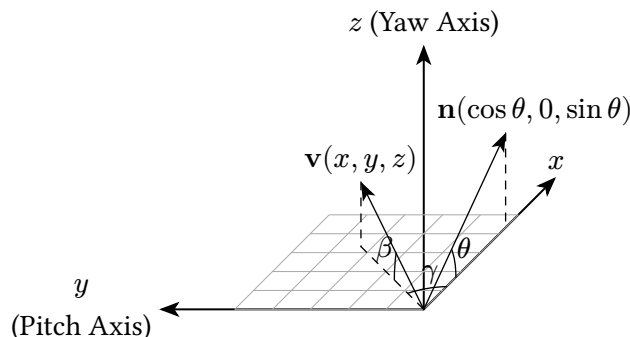


图 6 在 YawLink 坐标系下计算控制误差

在 YawLink 坐标系下， z 轴和 y 轴分别是云台的 Yaw 轴和 Pitch 轴。求控制误差，只需依次求将 $\mathbf{n}$ 变换到 $\mathbf{v}$ 所需要在 Yaw 和 Pitch 轴旋转的角度。

对于 Yaw 轴控制误差，显然 $\text{err}_{\text{yaw}} = \gamma = \tan^{-1}\left(\frac{y}{x}\right)$ 。

对于 Pitch 轴控制误差，设 $x' = \sqrt{x^2 + y^2}$ ，有：

$$\begin{aligned} -\text{err}_{\text{pitch}} &= \text{target} - \text{current} \\ &= \beta - \theta \\ &= \tan^{-1}\left(\frac{z}{x'}\right) - \tan^{-1}\left(\frac{\sin \theta}{\cos \theta}\right) \\ &= \tan^{-1}\left(\frac{z \cos \theta - x' \sin \theta}{z \sin \theta + x' \cos \theta}\right) \end{aligned}$$

5.2. 全向底盘控制器

约定复数 $a + bi$ (i 为虚数单位) 表示向量 (a, b) 。

底盘控制器接收底盘的目标水平控制速度 $\dot{\mathbf{r}} = \dot{x} + i\dot{y}$ 和目标旋转控制速度 $\dot{\theta}$ ，并输出轮电机的目标控制力矩。

5.2.1. 麦轮底盘建模

麦轮底盘等效于长方形排布的全向底盘。

在水平面上，以麦轮底盘的中心为原点，从原点出发，平行于任意两相邻轮的连线，构造 x 轴，建立平面直角坐标系，为自然坐标系。

从 x 轴开始，逆时针依次遍历位于第一、第二、第三、第四象限处的轮，分别记其编号 i 为 1, 2, 3, 4。

轮电机输出轴转速的正方向，以拇指指向与原点向外的射线同向的右手四指方向决定。

对于任意轮电机 i ，设轮半径为 r ，观测其输出轴转速为 ω_i 。若不考虑打滑，底盘在此处相对地面速度的可观测分量为 $v_i = -\omega_i r$ （垂直于全向轮的速度分量不可观测）。

对于任意轮 i ，不难计算 v_i 的正方向：

$$\mathbf{p}_i = \begin{cases} -1 + i, & i = 1 \\ -1 - i, & i = 2 \\ 1 - i, & i = 3 \\ 1 + i, & i = 4 \end{cases}$$

设 $t = 0$ 时刻，轮中心位置向量为 $(a, b > 0)$ ：

$$\mathbf{c}_i = \begin{cases} a + bi, & i = 1 \\ -a + bi, & i = 2 \\ -a - bi, & i = 3 \\ a - bi, & i = 4 \end{cases}$$

5.2.2. 全向底盘建模

全向底盘（通常为正方形排布）可认为是麦轮底盘长短边相等，即 $a = b$ 时的特例。

5.2.3. 观测底盘速度

每帧解算中，均认为当前时刻为 $t = 0$ 时刻。

以 $t = 0$ 时刻的自然坐标系为基准，建立世界坐标系。

对于底盘，其世界坐标系下有三个自由度 x, y, θ （中心位置、旋转角），设 $\mathbf{r} = x + iy$ 。

特别地，记 $t = 0$ 时刻三自由度的值分别为 (x_0, y_0, θ_0) ，显然此时刻 $(x_0, y_0, \theta_0) = 0$ 。

对于轮 i ，在任意 t 时刻，其中心位置向量为： $\mathbf{r}_i(t) = x + iy + \mathbf{c}_i e^{i\theta}$

求导得任意 t 时刻，轮 i 的中心速度向量： $\dot{\mathbf{r}}_i(t) = \dot{x} + i\dot{y} + i\dot{\theta}\mathbf{c}_i e^{i\theta}$

于是 $t = 0$ 时刻：

$$\dot{\mathbf{r}}_i(0) = \dot{x}_0 + i\dot{y}_0 + i\dot{\theta}_0\mathbf{c}_i$$

将 $\dot{\mathbf{r}}_i(0)$ 投影到 v_i 的正方向，得到底盘在轮 i 处的可观测速度分量：

$$\dot{r}_i(0)_{\text{observable}} = \dot{\mathbf{r}}_i(0) \cdot \frac{\mathbf{p}_i}{\sqrt{2}}$$

展开为实数形式：

$$\dot{r}_i(0)_{\text{observable}} = \frac{1}{\sqrt{2}} \begin{cases} -\dot{x}_0 + \dot{y}_0 + (a+b)\dot{\theta}_0, & i = 1 \\ -\dot{x}_0 - \dot{y}_0 + (a+b)\dot{\theta}_0, & i = 2 \\ \dot{x}_0 - \dot{y}_0 + (a+b)\dot{\theta}_0, & i = 3 \\ \dot{x}_0 + \dot{y}_0 + (a+b)\dot{\theta}_0, & i = 4 \end{cases}$$

构造观测方程组：

$$\dot{r}_i(0)_{\text{observable}} = v_i, \quad i = 1, 2, 3, 4$$

共 4 个方程，和 3 个未知数，显然这是一个超定方程组，将其化为矩阵形式 $A\mathbf{x} = \mathbf{b}$ ：

$$A = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 & 1 & a+b \\ -1 & -1 & a+b \\ 1 & -1 & a+b \\ 1 & 1 & a+b \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} \dot{x}_0 \\ \dot{y}_0 \\ \dot{\theta}_0 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{pmatrix}$$

构造正规方程 $A^\top A\mathbf{x} = A^\top \mathbf{b}$ ，解之，得到最小二乘近似解：

$$\begin{pmatrix} \dot{x}_0 \\ \dot{y}_0 \\ \dot{\theta}_0 \end{pmatrix} = \frac{\sqrt{2}}{4} \begin{pmatrix} -v_1 - v_2 + v_3 + v_4 \\ v_1 - v_2 - v_3 + v_4 \\ \frac{1}{a+b}(v_1 + v_2 + v_3 + v_4) \end{pmatrix}$$

用轮的输出轴转速 ω_i 表示：

$$\begin{pmatrix} \dot{x}_0 \\ \dot{y}_0 \\ \dot{\theta}_0 \end{pmatrix} = -\frac{\sqrt{2}r}{4} \begin{pmatrix} -\omega_1 - \omega_2 + \omega_3 + \omega_4 \\ \omega_1 - \omega_2 - \omega_3 + \omega_4 \\ \frac{1}{a+b}(\omega_1 + \omega_2 + \omega_3 + \omega_4) \end{pmatrix}$$

即得到底盘的观测速度 $(\dot{x}_0, \dot{y}_0, \dot{\theta}_0)$ 。

5.2.4. 简单闭环控制

假定底盘功率无限，电机能提供的扭矩无限，我们可以简单地对观测速度的平移和旋转分量进行闭环控制。

通过 PID 控制器，基于底盘的目标控制速度 $(\dot{x}, \dot{y}, \dot{\theta})$ 与观测速度 $(\dot{x}_0, \dot{y}_0, \dot{\theta}_0)$ ，闭环底盘的目标控制加速度 $(\ddot{x}, \ddot{y}, \ddot{\theta})$ 。

$$\ddot{x} = \text{PID}_{\text{velocity}}(\dot{x} - \dot{x}_0),$$

$$\ddot{y} = \text{PID}_{\text{velocity}}(\dot{y} - \dot{y}_0),$$

$$\ddot{\theta} = \text{PID}_{\text{velocity}}(\dot{\theta} - \dot{\theta}_0).$$

5.2.5. 计算电机控制力矩

设轮 i 在接地点处对底盘的目标作用力为 F_i 。对底盘，由牛顿第二定律：

$$\frac{1}{\sqrt{2}}(-F_1 - F_2 + F_3 + F_4) = m\ddot{x}$$

$$\frac{1}{\sqrt{2}}(F_1 - F_2 - F_3 + F_4) = m\ddot{y}$$

$$\frac{1}{\sqrt{2}} \sum_{i=1}^4 \mathbf{c}_i \times F_i \mathbf{p}_i = \frac{a+b}{\sqrt{2}}(F_1 + F_2 + F_3 + F_4) = J\ddot{\theta}$$

写为矩阵形式 $A\mathbf{x} = \mathbf{b}$ ：

$$A = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 & -1 & 1 & 1 \\ 1 & -1 & -1 & 1 \\ a+b & a+b & a+b & a+b \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} m\ddot{x} \\ m\ddot{y} \\ J\ddot{\theta} \end{pmatrix}$$

共 3 个方程，和 4 个未知数，显然这是一个欠定方程组，若不增加条件无法求解。

要求最节省能量的解，显然需要使 F_i 的范数和 $\sum_{i=1}^4 \|F_i\|$ 最小，即最小范数解。这可通过 Moore-Penrose 伪逆求得：

$$\mathbf{x} = \begin{pmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{pmatrix} = A^\top (AA^\top)^{-1} \mathbf{b} = \frac{\sqrt{2}}{4} \begin{pmatrix} -m\ddot{x} + m\ddot{y} + \frac{J\ddot{\theta}}{a+b} \\ -m\ddot{x} - m\ddot{y} + \frac{J\ddot{\theta}}{a+b} \\ m\ddot{x} - m\ddot{y} + \frac{J\ddot{\theta}}{a+b} \\ m\ddot{x} + m\ddot{y} + \frac{J\ddot{\theta}}{a+b} \end{pmatrix}$$

写为复数形式：

$$F_i = \frac{\sqrt{2}}{4} \left(m\mathbf{p}_i \cdot \ddot{\mathbf{r}} + \frac{J}{a+b} \ddot{\theta} \right)$$

于是轮的目标控制力矩：

$$\tau_{i \text{ feedforward}} = -rF_i = -\frac{\sqrt{2}r}{4} \left(m\mathbf{p}_i \cdot \ddot{\mathbf{r}} + \frac{J}{a+b} \ddot{\theta} \right)$$

不妨设 $\tau_r = -\frac{\sqrt{2}mr}{4} \ddot{\mathbf{r}}$, $\tau_\theta = -\frac{\sqrt{2}Jr}{4(a+b)} \ddot{\theta}$, 表达式可简化为：

$$\tau_{i \text{ feedforward}} = \mathbf{p}_i \cdot \tau_r + \tau_\theta$$

设 $\hat{\tau}_r = \frac{\tau_r}{\|\tau_r\|}$, $\tau_r = \|\tau_r\|$, 则 $\mathbf{p}_i \cdot \tau_r = \mathbf{p}_i \cdot \hat{\tau}_r \tau_r$ 。再设 $\lambda_i = \mathbf{p}_i \cdot \hat{\tau}_r$, 表达式可继续简化：

$$\tau_{i \text{ feedforward}} = \lambda_i \tau_r + \tau_\theta$$

在计算目标控制力矩时引入了最小范数假设，在遇到扰动（如打滑）时抗干扰能力不佳。考虑引入速度环 PID 控制器，将底盘观测速度 $(\dot{x}_0, \dot{y}_0, \dot{\theta}_0)$ 下，理论计算的底盘在轮 i 处的可观测速度分量作为控制速度，原计算得到的控制力矩作为 PID 控制器的前馈。

$$\tau_{i \text{ PID}} = \text{PID}_{\text{velocity}} \left(\begin{matrix} \omega_i \\ \text{expected} \end{matrix} - \omega_i \right) = \text{PID}_{\text{velocity}} \left(\begin{matrix} \text{observable} \\ \dot{\hat{r}}_i(0) \\ r \end{matrix} - \omega_i \right)$$

于是总的目标控制力矩：

$$\begin{aligned} \tau_i &= \tau_{i \text{ feedforward}} + \tau_{i \text{ PID}} \\ &= \lambda_i \tau_r + \tau_\theta + \tau_{i \text{ PID}} \end{aligned}$$

τ_r 和 τ_θ 为未约束的目标控制力矩。在同一次解算中， λ_i 和 $\tau_{i \text{ PID}}$ 恒不变，认为其为常数。

5.2.6. 增加约束

前文由 PID 控制器计算了底盘的目标控制加速度和目标控制力矩，但显然底盘功率并非无限，电机能提供的扭矩也并非无限，底盘在加速度过大时也会存在打滑，因此实际控制加速度不可能总与期望相符，考虑引入约束条件以保证控制力矩合法。

5.2.6.1. 控制力矩约束

不难发现 τ_i 仅与 τ_r, τ_θ 相关。可将问题转化为线性规划 (LP) 问题，决策变量为 τ_t, τ_r 。通过极大化目标函数 $z(\ddot{r}, \ddot{\theta}) = \alpha \ddot{r} + \beta \ddot{\theta}$ 以确保获得最优解。

对于控制力矩, 我们要求它小于等于上文计算的待约束的目标控制力矩, 同时必须大于等于 0。用 $\tau_{r \max}, \tau_{\theta \max}$ 表示上文计算的未约束的目标控制力矩, 有³:

$$\begin{cases} 0 \leq \tau_r \leq \tau_{r \max} \\ 0 \leq \tau_{\theta} \leq \tau_{\theta \max} \end{cases}$$

5.2.6.2. 打滑约束

单个电机的控制力矩 τ 总有上限。同时为防止打滑而浪费功率, 电机的输出力矩也应限制在一个较小范围。

由于电机的最大控制力矩几乎总是大于使轮发生滑动所需的力矩, 因此仅考虑使轮不发生打滑, 即满足 $|F| = \left| \frac{\tau}{r} \right| \leq \mu F_N$ 。

假定车重均匀分布于每个轮, 得约束不等式:

$$|\lambda_i \tau_r + \tau_{\theta}| \leq \frac{\mu mgr}{4}, \quad i = 1, 2, 3, 4$$

由 $\mathbf{p}_1 = -\mathbf{p}_3$, 得 $\lambda_1 = \mathbf{p}_1 \cdot \hat{\boldsymbol{\tau}}_r = -\mathbf{p}_3 \cdot \hat{\boldsymbol{\tau}}_r = -\lambda_3$, 同理可证 $\lambda_2 = -\lambda_4$ 。

若设 $\begin{cases} x = \tau_t \\ y = \tau_r \end{cases}$, 易证该不等式的可行域一定是沿 x, y 轴对称的菱形, 其上顶点恒为 $(0, \frac{\mu mgr}{4})$, 右顶点恒为 $(\frac{\mu mgr}{4 \max(|\lambda_1|, |\lambda_2|)}, 0)$ 。

5.2.6.3. 功率约束

显然底盘功率存在上限, 需要加以限制。

单个电机预期消耗功率的经验公式:

$$P_i(\tau_i) = k_1 \tau_i^2 + \tau_i \omega_i + k_2 \omega_i^2$$

其中, k_1 (电流热损耗系数) 与 k_2 (机械损耗系数) 为常数, 由实验拟合得出。

代入 $\tau_i = \lambda_i \tau_r + \tau_{\theta} + \tau_{i \text{ PID}}$, 得到:

$$P_{\text{total}} = \sum_{i=1}^4 P_i(\tau_i) = A \tau_r^2 + B \tau_r \tau_{\theta} + C \tau_{\theta}^2 + D \tau_r + E \tau_{\theta} + F \leq P_{\max}$$

其中:

$$A = 4k_1$$

$$B = 0$$

$$C = 4k_1$$

$$D = \lambda_1 \left(\omega_1 - \omega_3 + 2k_1 \left(\tau_{1 \text{ PID}} - \tau_{3 \text{ PID}} \right) \right) + \lambda_2 \left(\omega_2 - \omega_4 + 2k_1 \left(\tau_{2 \text{ PID}} - \tau_{4 \text{ PID}} \right) \right)$$

$$E = 2k_1 \sum_{\text{PID}} \tau_i + \sum \omega_i$$

$$F = k_1 \sum_{\text{PID}} \tau_i^2 + \sum_{\text{PID}} \tau_i \omega_i + k_2 \sum \omega_i^2$$

显然其圆锥曲线判别式 $B^2 \leq 4AC$, 该不等式的可行域为一个圆。

³ $\tau_{\theta \max}$ 可能为负, 本文只考虑其非负的情况。在实际的算法实现中, 若遇到负值, 将相关值取相反数处理, 完成求解后, 将结果反向映射回控制量。

由于圆不满足线性规划问题的定义，该问题转化为非线性规划问题 (NLP)。NLP 问题是难以求解的 (NP-Hard)，考虑利用表达式的良好性质简化问题。

首先，注意到约束表达式是一个二次不等式组，即二次约束，且除该约束外，其它约束均为线性。又注意到 P_{total} 是一个圆，显然圆为凸函数，于是相应的二次约束为凸二次约束，因此问题是一个仅有一个凸二次约束的凸二次约束规划 (QCP) 问题。

5.2.7. 问题求解

问题的数学形式为：

$$\begin{aligned} & \text{maximize} \quad z(\tau_r, \tau_\theta) = \alpha\tau_t + \beta\tau_r \\ & \text{subject to} \quad 0 \leq \tau_r \leq \tau_{r \max} \\ & \quad \quad \quad 0 \leq \tau_\theta \leq \tau_{\theta \max} \\ & \quad \quad \quad |\lambda_i \tau_r + \tau_\theta| \leq \frac{\mu mgr}{4} \quad (i = 1, 2, 3, 4) \\ & \quad \quad \quad P_{\text{total}} = \sum_{i=1}^4 P_i(\tau_i) \leq P_{\max} \end{aligned}$$

该规划问题的形式，与舵轮底盘控制器的规划问题，形式上完全相同（且更简单）。因此两算法使用同一求解器，具体求解过程可见小节 5.3.6.2。

5.3. 舵轮底盘控制器

舵轮底盘控制器接收底盘的目标水平控制速度 $\dot{\mathbf{r}} = \dot{x} + i\dot{y}$ 和目标旋转控制速度 $\dot{\theta}$ ，并输出轮电机和舵电机的目标控制力矩。

5.3.1. 舵轮底盘建模

对于舵轮底盘，设其质量为 m ，转动惯量为 J ，轮的几何中心与底盘中心的水平距离均为 R 。假定底盘为刚体，质量中心位于底盘中心。

对于舵轮，设轮半径为 r ，轮与地面的静摩擦系数为 μ 。假定轮为轻质刚性轮，无质量，无转动惯量，与地面不发生滑动摩擦，轮的 z 方向（舵向）旋转轴穿过轮的几何中心。忽略轮的宽度，即认为 z 方向（舵向）的旋转是无摩擦的。

以舵轮底盘的中心为原点，从原点出发，逆时针依次选择两相邻轮，构成的射线为 x, y 轴，建立自然坐标系。

逆时针依次遍历位于 x 正， y 正， x 负， y 负处的轮，分别记其编号 i 为 1, 2, 3, 4。

对于任意轮 i ，将右手拇指指向轮电机输出轴方向，定义右手四指方向为轮转速 ω_i 的正方向。令轮以速度 $\omega_i > 0$ 旋转，定义轮中心正上方的的线速度方向为舵的正方向。设轮的正方向与 x 轴的夹角为 α_i ，轮中心与底盘中心连线与 x 轴夹角为：

$$\varphi_i = \begin{cases} 0, & i = 1, \\ \pi/2, & i = 2, \\ \pi, & i = 3, \\ 3\pi/2, & i = 4. \end{cases}$$

若观测其输出轴转速为 ω_i 。若不考虑打滑，底盘在此处相对地面的速度大小为 $u_i = \omega_i r$ ，速度向量为 $\mathbf{u}_i = \omega_i r e^{i\alpha_i}$ 。

5.3.2. 观测底盘速度

以 $t = 0$ 时刻的自然坐标系为基准，建立世界坐标系。

在世界坐标系下，底盘有三个自由度 x, y, θ 。显然 $t = 0$ 时刻 $(x, y, \theta) = (x_0, y_0, \theta_0) = 0$ 。

对于轮 i ，在任意 t 时刻，其中心位置向量为： $\mathbf{r}_i(t) = x + iy + Re^{i(\theta + \varphi_i)}$ 。

其中心速度向量为： $\mathbf{v}_i(t) = \dot{x} + i\dot{y} + iRe^{i(\theta + \varphi_i)}\dot{\theta}$ 。

令 $t = 0$ ，则轮中心速度向量为： $\mathbf{v}_i(0) = \dot{x}_0 + i\dot{y}_0 + iRe^{i\varphi_i}\dot{\theta}_0$ 。

为了求解该时刻的底盘实际速度 $\dot{x}_0, \dot{y}_0, \dot{\theta}_0$ ，代入速度观测量，可得方程组：

$$\{\mathbf{v}_i(0) = \mathbf{u}_i, \quad i = 1, 2, 3, 4$$

展开为实数形式：

$$\begin{cases} \dot{x}_0 - R\dot{\theta}_0 \sin(\varphi_i) = r\omega_i \cos(\alpha_i) \\ \dot{y}_0 + R\dot{\theta}_0 \cos(\varphi_i) = r\omega_i \sin(\alpha_i) \end{cases}, \quad i = 1, 2, 3, 4$$

继续展开：

$$\begin{pmatrix} \dot{x}_0 & \dot{y}_0 + R\dot{\theta}_0 \\ \dot{x}_0 - R\dot{\theta}_0 & \dot{y}_0 \\ \dot{x}_0 & \dot{y}_0 - R\dot{\theta}_0 \\ \dot{x}_0 + R\dot{\theta}_0 & \dot{y}_0 \end{pmatrix} = r \begin{pmatrix} \omega_1 \cos(\alpha_1) & \omega_1 \sin(\alpha_1) \\ \omega_2 \cos(\alpha_2) & \omega_2 \sin(\alpha_2) \\ \omega_3 \cos(\alpha_3) & \omega_3 \sin(\alpha_3) \\ \omega_4 \cos(\alpha_4) & \omega_4 \sin(\alpha_4) \end{pmatrix}$$

共 8 个方程，和 3 个未知数，显然这也是一个超定方程组，可以通过构造特征方程，求最小二乘近似解：

$$\begin{aligned} \dot{x}_0 &= \frac{r}{4}(\omega_1 \cos \alpha_1 + \omega_2 \cos \alpha_2 + \omega_3 \cos \alpha_3 + \omega_4 \cos \alpha_4), \\ \dot{y}_0 &= \frac{r}{4}(\omega_1 \sin \alpha_1 + \omega_2 \sin \alpha_2 + \omega_3 \sin \alpha_3 + \omega_4 \sin \alpha_4), \\ \dot{\theta}_0 &= -\frac{r}{4R}(-\omega_1 \sin \alpha_1 + \omega_2 \cos \alpha_2 + \omega_3 \sin \alpha_3 - \omega_4 \cos \alpha_4). \end{aligned}$$

得到底盘的观测速度。

5.3.3. 闭环底盘速度

通过 PID 控制器，基于底盘的目标控制速度 $(\dot{x}, \dot{y}, \dot{\theta})$ 与观测速度 $(\dot{x}_0, \dot{y}_0, \dot{\theta}_0)$ ，闭环底盘的目标控制加速度 $(\ddot{x}, \ddot{y}, \ddot{\theta})$ 。

$$\begin{aligned} \ddot{x} &= \text{PID}_{\text{velocity}}(\dot{x} - \dot{x}_0), \\ \ddot{y} &= \text{PID}_{\text{velocity}}(\dot{y} - \dot{y}_0), \\ \ddot{\theta} &= \text{PID}_{\text{velocity}}(\dot{\theta} - \dot{\theta}_0). \end{aligned}$$

5.3.4. 计算电机控制力矩

5.3.4.1. 轮电机

在每个控制帧内, 设底盘以观测速度为初速度, 做加速度为目标控制加速度的匀加速运动, 则在每个控制帧内, $(\dot{x}_0, \dot{y}_0, \dot{\theta}_0) = \text{const}$, $(\ddot{x}, \ddot{y}, \ddot{\theta}) = \text{const}$, 有:

$$\begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = \begin{pmatrix} \dot{x}_0 + \ddot{x}t \\ \dot{y}_0 + \ddot{y}t \\ \dot{\theta}_0 + \ddot{\theta}t \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} = \begin{pmatrix} \dot{x}_0 t + \frac{1}{2}\ddot{x}t^2 \\ \dot{y}_0 t + \frac{1}{2}\ddot{y}t^2 \\ \dot{\theta}_0 t + \frac{1}{2}\ddot{\theta}t^2 \end{pmatrix}$$

于是 t 时刻轮中心位置向量:

$$\mathbf{r}_i(t) = \dot{x}_0 t + \frac{1}{2}\ddot{x}t^2 + i\left(\dot{y}_0 t + \frac{1}{2}\ddot{y}t^2\right) + Re^{i(\varphi_i + \dot{\theta}_0 t + \frac{1}{2}\ddot{\theta}t^2)}$$

t 时刻轮中心速度向量:

$$\dot{\mathbf{r}}_i(t) = \dot{x}_0 + \ddot{x}t + i(\dot{y}_0 + \ddot{y}t) + iRe^{i(\varphi_i + \dot{\theta}_0 t + \frac{1}{2}\ddot{\theta}t^2)}(\dot{\theta}_0 + \ddot{\theta}t)$$

在自然坐标系下, 对底盘做受力分析。

由于底盘做匀加速运动, 坐标系非惯性系, 由达朗贝尔原理, 可将加速度设为假想的惯性力。

在 $t = 0$ 时刻, 底盘受平移力:

$$\mathbf{F} = F_x + iF_y = -m\ddot{x} - im\ddot{y}$$

底盘受旋转力矩:

$$M = -J\ddot{\theta}$$

在该坐标系下, 底盘受假想的惯性力但保持静止。每个轮可提供平行于轮方向的主动力和垂直于轮方向的静摩擦力, 约束底盘保持静止。

轮无论位于何方向, 其主动力和静摩擦力总是互相垂直, 若假定无论如何都不会发生滑动, 可以将 4 个轮当成光滑铰链约束, 提供任意方向和大小的约束反力阻止底盘移动, 在解出约束反力后再分解为轮的主动力和静摩擦力。铰链约束仅提供反力, 不提供反力矩。设每个铰链约束的约束反力向量为 $F_{xi} + iF_{yi}$, 可以列出方程:

$$\begin{cases} \mathbf{F} + \sum_{i=1}^4 (F_{xi} + iF_{yi}) = 0 \\ M + \sum_{i=1}^4 (-RF_{xi} \sin(\varphi_i) + RF_{yi} \cos(\varphi_i)) = 0 \end{cases}$$

展开方程得到:

$$\begin{cases} F_{x1} + F_{x2} + F_{x3} + F_{x4} + F_x = 0 \\ F_{y1} + F_{y2} + F_{y3} + F_{y4} + F_y = 0 \\ R(F_{y1} - F_{x2} - F_{y3} + F_{x4}) + M = 0 \end{cases}$$

共 3 个实数方程，但存在 8 个未知量。显然这也是一个欠定方程，若不增加条件无法求解。考虑引入最小范数条件，即最小化反力总平方和：

$$\min \sum_{i=1}^4 (F_{xi}^2 + F_{yi}^2)$$

构建拉格朗日函数，引入乘数 $\lambda_1, \lambda_2, \lambda_3$ ：

$$L = \sum (F_{xi}^2 + F_{yi}^2) + \lambda_1 \left(\sum F_{xi} + F_x \right) + \lambda_2 \left(\sum F_{yi} + F_y \right) + \lambda_3 \left(F_{y1} - F_{x2} - F_{y3} + F_{x4} + \frac{M}{R} \right)$$

对每个变量求偏导并令其为零，解得：

$$\begin{aligned} F_{x1} &= -\frac{\lambda_1}{2}, & F_{x2} &= \frac{-\lambda_1 + \lambda_3}{2}, & F_{x3} &= -\frac{\lambda_1}{2}, & F_{x4} &= \frac{-\lambda_1 - \lambda_3}{2}, \\ F_{y1} &= \frac{-\lambda_2 - \lambda_3}{2}, & F_{y2} &= -\frac{\lambda_2}{2}, & F_{y3} &= \frac{-\lambda_2 + \lambda_3}{2}, & F_{y4} &= -\frac{\lambda_2}{2}. \end{aligned}$$

将上述表达式代入平衡方程，解得：

$$\lambda_1 = \frac{F_x}{2}, \quad \lambda_2 = \frac{F_y}{2}, \quad \lambda_3 = \frac{M}{2R}.$$

代回各反力表达式，得到：

$$\begin{pmatrix} \mathbf{F}_1 \\ \mathbf{F}_2 \\ \mathbf{F}_3 \\ \mathbf{F}_4 \end{pmatrix} = \begin{pmatrix} F_{x1} & F_{y1} \\ F_{x2} & F_{y2} \\ F_{x3} & F_{y3} \\ F_{x4} & F_{y4} \end{pmatrix} = \frac{1}{4} \begin{pmatrix} m\ddot{x} & m\ddot{y} + \frac{J\ddot{\theta}}{R} \\ m\ddot{x} - \frac{J\ddot{\theta}}{R} & m\ddot{y} \\ m\ddot{x} & m\ddot{y} - \frac{J\ddot{\theta}}{R} \\ m\ddot{x} + \frac{J\ddot{\theta}}{R} & m\ddot{y} \end{pmatrix}$$

写为复数形式：

$$\mathbf{F}_i = \frac{m\ddot{x} + im\ddot{y} + i\frac{J\ddot{\theta}}{R}e^{i\varphi_i}}{4}$$

设舵轮 i 的正方向相对底盘的夹角（舵向角）的观测值为 ζ_{i0} 。

将 $\mathbf{F}_i$ 投影到 ζ_{i0} 方向，乘以轮半径 r ，得到轮的目标控制力矩：

$$\tau_{i \text{ feedforward}} = r(\mathbf{F}_i \cdot e^{i\zeta_{i0}}) = r \left(\frac{m\ddot{x}R \cos(\zeta_{i0}) + m\ddot{y}R \sin(\zeta_{i0}) + J\ddot{\theta} \sin(\zeta_{i0} - \varphi_i)}{4R} \right)$$

在计算目标控制力矩时引入了较多假设（如最小范数条件），在遇到扰动（如打滑）时抗干扰能力不佳。考虑引入速度环 PID 控制器，将底盘观测速度 $(\dot{x}_0, \dot{y}_0, \dot{\theta}_0)$ 下理论计算的轮速度，在轮正方向上的投影作为控制速度，原计算得到的控制力矩作为 PID 控制器的前馈。

$$\begin{aligned} \tau_i &= \tau_{i \text{ feedforward}} + \text{PID}_{\text{velocity}} \left(\frac{\dot{\mathbf{r}}_i(0) \cdot e^{i\zeta_{i0}}}{r} - \omega_i \right) \\ &= r \left(\frac{m\ddot{x}R \cos(\zeta_{i0}) + m\ddot{y}R \sin(\zeta_{i0}) + J\ddot{\theta} \sin(\zeta_{i0} - \varphi_i)}{4R} \right) + \\ &\quad \text{PID}_{\text{velocity}} \left(\frac{\dot{x}_0 \cos(\zeta_{i0}) + \dot{y}_0 \sin(\zeta_{i0}) + R\dot{\theta}_0 \sin(\zeta_{i0} - \varphi_i)}{r} - \omega_i \right) \end{aligned}$$

5.3.4.2. 舵电机

设舵轮 i 的正方向相对底盘的夹角（舵向角）的目标控制值为 ζ_i ，则⁴：

$$\zeta_i(t) = \arg(\dot{\mathbf{r}}_i) - \theta$$

$$\dot{\zeta}_i(t) = \frac{\partial \zeta_i}{\partial t}$$

$\ddot{\zeta}_i(t)$ 的表达式过于复杂，由于轮为轻质轮，简单起见可令 $\zeta_i = 0$ ，即直接由 PID 控制器闭环舵向角速度，不使用舵向角加速度作为前馈。

$t = 0$ 时刻：

$$\begin{aligned} \zeta_i(0) &= \tan^{-1}(\dot{x}_0 - R\dot{\theta}_0 \sin(\varphi_i), \dot{y}_0 + R\dot{\theta}_0 \cos(\varphi_i)) \\ \dot{\zeta}_i(0) &= \frac{\dot{x}_0 \ddot{y} - \dot{y}_0 \ddot{x} - \dot{\theta}_0 (\dot{x}_0^2 + \dot{y}_0^2) + R \cos(\varphi_i) (\ddot{\theta} \dot{x}_0 - \dot{\theta}_0 (\ddot{x} + \dot{\theta}_0 \dot{y}_0)) + R \sin(\varphi_i) (\ddot{\theta} \dot{y}_0 - \dot{\theta}_0 (\ddot{y} + \dot{\theta}_0 \dot{x}_0))}{(\dot{x}_0 - R\dot{\theta}_0 \sin(\varphi_i))^2 + (\dot{y}_0 + R\dot{\theta}_0 \cos(\varphi_i))^2} \end{aligned}$$

注意到上述表达式在轮 i 处线速度 $\dot{\mathbf{r}}_i = 0$ ，即 $\begin{cases} \dot{x}_0 - R\dot{\theta}_0 \sin(\varphi_i) = 0 \\ \dot{y}_0 + R\dot{\theta}_0 \cos(\varphi_i) = 0 \end{cases}$ 时无法求值。

对 $\zeta_i(0)$ ，显然极限 $\lim_{\substack{\dot{x}_0 \rightarrow R\dot{\theta}_0 \sin(\varphi_i) \\ \dot{y}_0 \rightarrow -R\dot{\theta}_0 \cos(\varphi_i)}} \zeta_i(0)$ 不定，考虑将奇异点代入 $\zeta_i(t)$ ：

$$\zeta_i(t) \xrightarrow[\substack{\dot{x}_0 \mapsto R\dot{\theta}_0 \sin(\varphi_i) \\ \dot{y}_0 \mapsto -R\dot{\theta}_0 \cos(\varphi_i)}]{} \zeta_i'(t)$$

对 t 求极限得：

$$\begin{aligned} \zeta_i(0) &= \lim_{\substack{\dot{\mathbf{r}}_i=0 \\ t \rightarrow 0^+}} \zeta_i'(t) \\ &= \tan^{-1}(\ddot{x} - R(\ddot{\theta} \sin(\varphi_i) + \dot{\theta}_0^2 \cos(\varphi_i)), \ddot{y} + R(\ddot{\theta} \cos(\varphi_i) - \dot{\theta}_0^2 \sin(\varphi_i))) \end{aligned}$$

对 $\dot{\zeta}_i(0)$ ，显然 $\lim_{\substack{\dot{x}_0 \rightarrow R\dot{\theta}_0 \sin(\varphi_i) \\ \dot{y}_0 \rightarrow -R\dot{\theta}_0 \cos(\varphi_i)}} \dot{\zeta}_i(0)$ 同样不定，为简单起见，定义：

$$\dot{\zeta}_i \Big|_{\dot{\mathbf{r}}_i=0} = 0$$

表达式 $\zeta_i(0)$ 在在轮 i 处加速度 $\ddot{\mathbf{r}}_i = 0$ 时依旧无解，为简单起见，定义此时刻舵向控制力为 0。

合并上述表达式，得到舵向目标控制力矩：

$$\tau_{\zeta_i} = \begin{cases} \text{PID}_{\text{velocity}}(\dot{\zeta}_i(0) + \text{PID}_{\text{angle}}(\zeta_i(0) - \zeta_{i0}) - \dot{\zeta}_{i0}), & \dot{\mathbf{r}}_i \neq 0 \\ \text{PID}_{\text{velocity}}\left(\text{PID}_{\text{angle}}\left(\zeta_i(0) - \zeta_{i0}\right) - \dot{\zeta}_{i0}\right), & \dot{\mathbf{r}}_i = 0 \\ 0, & \ddot{\mathbf{r}}_i = 0 \end{cases}$$

⁴ $\zeta_i(t)$ 和 $\dot{\zeta}_i(t)$ 完整表达式见附录（TODO）。

5.3.5. 增加约束

在小节 5.3.3 中, 由 PID 控制器计算了底盘的目标控制加速度, 但显然底盘功率并非无限, 底盘在加速度过大时也会存在打滑。因此实际控制加速度不可能总与期望相符, 考虑引入约束条件以保证加速度合法, 通过极大化目标函数 $z(\ddot{r}, \ddot{\theta}) = a\ddot{r} + b\ddot{\theta}$ 以确保获得最优解。

小节 5.3.3 计算的未约束的目标控制加速度, 在本小节中用 $(\ddot{x}_{\max}, \ddot{y}_{\max}, \ddot{\theta}_{\max})$ 表示。

设 $\alpha = \tan^{-1}(\ddot{x}_{\max}, \ddot{y}_{\max})$ 是底盘目标平移控制加速度的方向角, 有:

$$\ddot{x} = \ddot{r} \cos(\alpha), \ddot{y} = \ddot{r} \sin(\alpha)$$

设 $\ddot{r}_{\max} = \sqrt{\ddot{x}_{\max}^2 + \ddot{y}_{\max}^2}$, 有:

$$\ddot{x}_{\max} = \ddot{r}_{\max} \cos(\alpha), \ddot{y}_{\max} = \ddot{r}_{\max} \sin(\alpha)$$

在一次解算内, α 不变, 认为其是已知量。

5.3.5.1. 控制加速度约束

对于约束后的平移控制加速度 $\ddot{r}$, 它应当小于等于未约束的平移控制加速度 $\ddot{r}_{\max}$ 。

对于约束后的旋转控制加速度 $\ddot{\theta}$, 它应当小于等于未约束的旋转控制加速度 $\ddot{\theta}_{\max}$ ⁵。

于是:

$$\begin{cases} \ddot{r} \leq \ddot{r}_{\max} \\ \ddot{\theta} \leq \ddot{\theta}_{\max} \end{cases}$$

问题转化为二维线性规划 (LP) 问题, 决策变量为 $\ddot{r}, \ddot{\theta}$ 。

5.3.5.2. 打滑约束

要求轮不发生滑动, 即满足 $|\mathbf{F}_i| \leq \mu F_N$ 。

假定车重均匀分布于每个轮, 则约束不等式:

$$|\mathbf{F}_i| \leq \frac{\mu mg}{4}, \quad i = 1, 2, 3, 4$$

整理得:

$$\left(m\ddot{r}R \cos(\alpha) - J\ddot{\theta} \sin(\varphi_i)\right)^2 + \left(m\ddot{r}R \sin(\alpha) + J\ddot{\theta} \cos(\varphi_i)\right)^2 \leq \mu^2 m^2 g^2 R^2, \quad i = 1, 2, 3, 4$$

显然表达式是一个二次不等式组, 由于其不满足线性规划问题的定义, 问题转换为非线性规划 (NLP) 问题。

NLP 问题是难以求解的, 考虑利用表达式的良好性质简化问题。

代入多组典型参数, 绘制不等式组的可行域, 发现可行域恒为对 x, y 轴对称的菱形, 且形状仅与 α 相关, 且在 $\alpha = \frac{1}{2}k\pi$ ($k \in \mathbb{Z}$) 时, 可行域为菱形; 在 $\alpha = \frac{1}{2}k\pi + \frac{1}{4}\pi$ ($k \in \mathbb{Z}$) 时, 可行域为面积最大的豆腐形; 在 α 为其余值时, 可行域介于菱形与面积最大的豆腐形之间。

图 7 可行域的形状仅与 α 相关

由于不等式可行域即使在面积最大时, 形状仍与菱形接近, 考虑尽量优化计算速度, 认为可行域恒为菱形。于是问题回到线性规划 (LP) 问题。

⁵ $\ddot{\theta}_{\max}$ 可能为负, 本文只考虑其非负的情况。在实际的算法实现中, 若遇到负值, 将相关值取相反数处理, 完成求解后, 将结果反向映射回控制量。

菱形的右顶点 $A(\mu g, 0)$, 上顶点 $B(0, \frac{mR}{J}\mu g)$, 写为线性不等式形式:

$$\begin{cases} \frac{1}{\mu g} \left(x + \frac{Jy}{mR} \right) \leq 1 \\ \frac{1}{\mu g} \left(x - \frac{Jy}{mR} \right) \leq 1 \\ \frac{1}{\mu g} \left(-x + \frac{Jy}{mR} \right) \leq 1 \\ \frac{1}{\mu g} \left(-x - \frac{Jy}{mR} \right) \leq 1 \end{cases}$$

5.3.5.3. 功率约束

显然底盘功率存在上限, 需要加以限制。

单个电机预期消耗功率的经验公式:

$$P_i(\tau_i) = k_1 \tau_i^2 + \omega_i \tau_i + k_2 \omega_i^2 \quad (5.1)$$

其中, k_1 (电流热损耗系数) 与 k_2 (机械损耗系数) 为常数, 由实验拟合得出。

小节 5.3.4.1 计算了轮电机的控制力矩:

$$\tau_i = \tau_{i, \text{feedforward}} + \tau_{i, \text{PID}}$$

其中:

$$\begin{aligned} \tau_{i, \text{feedforward}} &= r \left(\frac{m\ddot{r} \cos(\zeta_{i0} - \alpha)}{4} + \frac{J\ddot{\theta} \sin(\zeta_{i0} - \varphi_i)}{4R} \right) \\ \tau_{i, \text{PID}} &= \text{PID}_{\text{velocity}} \left(\frac{\dot{x}_0 \cos(\zeta_{i0}) + \dot{y}_0 \sin(\zeta_{i0}) + R\dot{\theta}_0 \sin(\zeta_{i0} - \varphi_i)}{r} - \omega_i \right) \end{aligned}$$

注意到 $\tau_{i, \text{PID}}$ 在一次解算中恒为常数, 可将 τ_i 整理为关于 $\ddot{r}, \ddot{\theta}$ 的多项式:

$$\tau_i = \frac{mr \cos(\zeta_{i0} - \alpha)}{4} \ddot{r} + \frac{Jr \sin(\zeta_{i0} - \varphi_i)}{4R} \ddot{\theta} + \tau_{i, \text{PID}}$$

代入式 (5.1), 得到:

$$P_{\text{total}} = \sum_{i=1}^4 P_i(\tau_i) = A\ddot{r}^2 + B\ddot{r}\ddot{\theta} + C\ddot{\theta}^2 + D\ddot{r} + E\ddot{\theta} + F \leq P_{\text{max}}$$

其中:

$$\begin{aligned} A &= \sum_{i=1}^4 \frac{1}{16} k_1 m^2 r^2 \cos^2(\zeta_{i0} - \alpha) \\ B &= \sum_{i=1}^4 \frac{1}{8} k_1 m J r^2 R^{-1} \cos(\zeta_{i0} - \alpha) \sin(\zeta_{i0} - \varphi_i) \\ C &= \sum_{i=1}^4 \frac{1}{16} k_1 J^2 r^2 R^{-2} \sin^2(\zeta_{i0} - \varphi_i) \\ D &= \sum_{i=1}^4 \frac{1}{4} m r \cos(\zeta_{i0} - \alpha) \left(2k_1 \tau_{i, \text{PID}} + \omega_i \right) \end{aligned}$$

$$E = \sum_{i=1}^4 \frac{1}{4} J r R^{-1} \sin(\zeta_{i0} - \varphi_i) \left(2k_{1 \text{ PID}} \tau_i + \omega_i \right)$$

$$F = \sum_{i=1}^4 k_{1 \text{ PID}} \tau_i + \omega_i \tau_i + k_{2 \text{ PID}} \omega_i^2$$

显然不等式同样是一个二次不等式，对应的约束为二次约束。由 Cauchy-Schwarz 不等式 (柯西不等式)：

$$\left(\sum_{i=1}^4 \cos(\zeta_{i0} - \alpha) \sin(\zeta_{i0} - \varphi_i) \right)^2 \leq \left(\sum_{i=1}^4 \cos^2(\zeta_{i0} - \alpha) \right) \left(\sum_{i=1}^4 \sin^2(\zeta_{i0} - \varphi_i) \right)$$

由 $k_1, m, J, r, R > 0$ ，得 $\frac{d^2 m^2 J^2}{64 R^2} > 0$ ，两边同乘该因子，整理得：

$$B^2 \leq 4AC$$

可知该二次约束对应的 Q 矩阵是半正定的， P_{total} 是一个凸函数，相应的二次约束为凸二次约束。

5.3.6. 问题求解

问题的数学形式为：

$$\begin{aligned} & \text{maximize} \quad z(\ddot{r}, \ddot{\theta}) = a\tau_t + b\tau_r \\ & \text{subject to} \quad \ddot{r} \leq \ddot{r}_{\max} \\ & \quad \quad \quad \ddot{\theta} \leq \ddot{\theta}_{\max} \\ & \quad \quad \quad \frac{1}{\mu g} \left(x + \frac{Jy}{mR} \right) \leq 1 \\ & \quad \quad \quad \frac{1}{\mu g} \left(x - \frac{Jy}{mR} \right) \leq 1 \\ & \quad \quad \quad \frac{1}{\mu g} \left(-x + \frac{Jy}{mR} \right) \leq 1 \\ & \quad \quad \quad \frac{1}{\mu g} \left(-x - \frac{Jy}{mR} \right) \leq 1 \\ & \quad \quad \quad P_{\text{total}} = \sum_{i=1}^4 P_i(\tau_i) \leq P_{\max} \end{aligned}$$

除最后一项约束 (功率约束) 为凸二次约束外，目标函数和其余约束均为线性，线性规划的可行域一定是凸集。因此，该问题是一个仅有一个凸二次约束的凸二次约束规划 (QCP) 问题。

5.3.6.1. 采用商业通用求解器 COPT 求解

QCP 问题通常使用内点法求解，可以选择开源或商用的通用求解器来求解 QCP 问题，但市面上大部分求解器主要针对二次规划 (QP) 问题求解，可以求解 QCP 问题的通用求解器较少。

尝试采用国产商业通用求解器杉树 (COPT) 对问题进行求解：

Cardinal Optimizer v7.2.7. Build date Apr 11 2025
Copyright Cardinal Operations 2025. All Rights Reserved

Setting parameter 'TimeLimit' to 0.1

Setting parameter 'Threads' to 1
Model fingerprint: 27e74570

Using Cardinal Optimizer v7.2.7 on Linux
Hardware has 11 cores and 22 threads. Using instruction set X86_AVX2
Maximizing a QCP problem

The original problem has:
4 rows, 2 columns and 8 non-zero elements
1 quadratic constraints
The presolved problem has:
6 rows, 5 columns and 15 non-zero elements
1 cones

Starting barrier solver using 1 thread

Problem info:
Range of matrix coefficients: [1e-01,1e+00]
Range of rhs coefficients: [5e-01,5e-01]
Range of bound coefficients: [1e+00,1e+00]
Range of cost coefficients: [4e-01,1e+00]

Factor info:
Number of dense columns: 0
Number of matrix entries: 2.100e+01
Number of factor entries: 2.100e+01
Number of factor flops: 7.200e+01

Iter	Primal.Obj	Dual.Obj	Compl	Primal.Inf	Dual.Inf
0	+0.00000000e+00	-4.00000000e+00	9.00e+00	1.00e+00	1.41e+00
1	-3.63528555e-01	-1.10418946e+00	1.53e+00	1.91e-01	2.70e-01
2	-3.94420420e-01	-5.63866533e-01	3.35e-01	4.64e-02	6.56e-02
3	-4.23270927e-01	-4.91048482e-01	1.25e-01	1.89e-02	2.67e-02
4	-3.96195440e-01	-4.04062794e-01	1.33e-02	2.05e-03	2.90e-03
5	-3.99948595e-01	-4.00449746e-01	8.47e-04	1.33e-04	1.88e-04
6	-4.00218257e-01	-4.00225109e-01	1.15e-05	1.80e-06	2.55e-06
7	-4.00219033e-01	-4.00220047e-01	1.70e-06	2.67e-07	3.78e-07
8	-4.00218793e-01	-4.00218849e-01	9.49e-08	1.59e-08	1.49e-08

Barrier status: OPTIMAL
Primal objective: -4.00218793e-01
Dual objective: -4.00218849e-01
Duality gap (abs/rel): 5.60e-08 / 5.60e-08
Primal infeasibility (abs/rel): 1.59e-08 / 1.59e-08
Dual infeasibility (abs/rel): 1.49e-08 / 1.49e-08

Postsolving

Solving finished
Status: Optimal Objective: 4.0021879328e-01 Iterations: 8
Solution: (-0.006806, 2.035123)
Elapsed time: 0.00296087s

查看日志可知,求解器在 9 次迭代后得出了最优解 (Optimal),经验证该解是正确的。但求解器单次计算耗时约 3 ms,大于许用的计算时间 1 ms,控制的实时性无法得到有效保证。

5.3.6.2. 编写求解器求解析解

由于问题是一个仅有一个凸二次约束的凸二次约束规划 (QCP) 问题,且维数仅为 2 维,考虑配合图解法,手动编写求解器求问题的解析解。

对于凸规划问题,局部最优解是它的全局最优解。因此,要求问题的全局最优解,只需沿着其边缘搜寻局部最优解。

5.3.6.2.1. 求线性可行域按逆时针排序的角点序列

设 $\begin{cases} x = \ddot{r} \\ y = \ddot{\theta} \end{cases}$, 将问题转换到二维笛卡尔坐标系下。

打滑约束的可行域是沿 x, y 轴对称的菱形。从其右顶点开始,逆时针记录其顶点序列为多边形 $P = \{P_1, P_2, P_3, P_4\}$ 。

使用 Sutherland-Hodgman 算法,遍历剩余约束条件,对多边形进行裁剪。算法一次裁剪的伪代码如下:

```
for (int i = 0; i < input_points.count; i += 1) do
    Point current_point = input_points[i];
    Point prev_point = input_points[(i - 1) % input_points.count];

    Point Intersecting_point = ComputeIntersection(prev_point, current_point,
clip_edge)

    if (current_point inside clip_edge) then
        if (prev_point not inside clip_edge) then
            output_points.add(Intersecting_point);
        end if
        output_points.add(current_point);

    else if (prev_point inside clip_edge) then
        output_points.add(Intersecting_point);
    end if
done
```

• 第一次裁剪:

P 作为初始序列 input_points , 条件 $\ddot{r} \leq \ddot{r}_{\max}$ 作为裁剪边 clip_edge 输入, 执行算法。

• 第二次裁剪:

将上一次裁剪的输出 output_points 作为初始序列 input_points , 条件 $\ddot{\theta} \leq \ddot{\theta}_{\max}$ 作为裁剪边 clip_edge 输入, 执行算法, 完成第二次裁剪。

5.3.6.2.2. 求凸二次约束的最优解

对于凸规划问题,局部最优解是它的全局最优解。因此,作为一个显著有效的优化,若凸二次约束的最优解在其余线性约束的可行域内,则可直接得出全局最优解为求凸二次约束的最优解。

由于二次约束是凸的，最优解一定在可行域边缘，可以把不等式转换为等式，使用拉格朗日乘数法求解最优解。

将凸二次约束写为矩阵形式：

$$\frac{1}{2}\mathbf{x}^T Q \mathbf{x} + \mathbf{p}^T \mathbf{x} + r = 0$$

其中⁶：

$$\begin{aligned}\mathbf{x} &= \begin{bmatrix} \ddot{r} \\ \ddot{\theta} \end{bmatrix}, \\ Q &= \begin{bmatrix} 2A & B \\ B & 2C \end{bmatrix}, \\ \mathbf{p} &= \begin{bmatrix} D \\ E \end{bmatrix}, \\ r &= F - P_{\max}.\end{aligned}\tag{5.2}$$

需要极大化的目标函数：

$$z(\mathbf{x}) = \mathbf{a} \cdot \mathbf{x}$$

其中：

$$\mathbf{a} = \begin{bmatrix} a \\ b \end{bmatrix}$$

写出拉格朗日函数：

$$L(\mathbf{x}, \lambda) = \mathbf{a}^T \mathbf{x} - \lambda \left(\frac{1}{2} \mathbf{x}^T Q \mathbf{x} + \mathbf{p}^T \mathbf{x} + r \right)$$

求梯度并令其为 0：

$$\nabla_{\mathbf{x}} L = \mathbf{a} - \lambda(Q\mathbf{x} + \mathbf{p}) = 0 \implies \mathbf{a} = \lambda(Q\mathbf{x} + \mathbf{p})$$

求解线性方程（假设 Q^{-1} 存在）：

$$\mathbf{x} = Q^{-1} \left(\frac{1}{\lambda} \mathbf{a} - \mathbf{p} \right)\tag{5.3}$$

代入约束条件并化简，得到⁷：

$$\lambda = \sqrt{\frac{\mathbf{a}^T Q^{-1} \mathbf{a}}{\mathbf{p}^T Q^{-1} \mathbf{p} - 2r}}$$

计算 λ 并代入式 (5.3)，得到凸二次约束的最优解。若最优解在角点序列多边形内，则直接返回最优解。

在求解过程中我们假定 Q^{-1} 存在，但小节 5.3.5.3 仅能证明 Q 是半正定矩阵。当 Q 不正定时，约束曲线将由椭圆退化为抛物线或分段线性结构（如两条线段），导致优化问题欠定， Q^{-1} 不存在。此时在退化方向（如抛物线开口方向）上，最优解可能趋向无穷大。

⁶若 $A < 0$ ，按式 (5.2) 赋值会导致 Q 为负定矩阵，此时需对 A, B, C, D, E, F 取相反数处理。

⁷实际上 λ 存在正负两解，对应使目标函数最大和最小时的取值。但由于 Q 为半正定矩阵，正解使目标函数取得最大值，故取正解。

考虑引入 Moore-Penrose 伪逆 Q^+ 替代 Q^{-1} 。然而当 Q 奇异时，伪逆会强制给出最小范数解：一方面，该解滤除了描述系统退化特性的零空间分量；另一方面，在有解方向上产生与实际物理意义不符的数值结果。

因此，本文采用正则化方法 $(Q + \varepsilon I)^{-1}$ ($\varepsilon \rightarrow 0^+$)，当 Q 奇异时，在对角线上引入微小扰动，使得矩阵可逆。该对角线扰动等效于在优化问题中引入 Tikhonov 正则项：

$$\min_{\mathbf{x}} \|\mathbf{x}\|^2 \quad \text{s.t. original constraints}$$

在退化方向上，扰动诱导的特征值偏移 ($\lambda_i \rightarrow \lambda_i + \varepsilon$) 使解的范数发散：

$$\mathbf{x}_{\text{reg}} \approx \frac{1}{\varepsilon} \mathbf{v}_{\text{null}} + O(1)$$

这种可控的发散特性正确符合了解要求，且当 $\varepsilon \rightarrow 0^+$ 时收敛于理论极限解。

5.3.6.2.3. 用二次约束裁剪多边形

来到这一步说明切点不在多边形内，因此最优解一定在多边形与二次约束的交点，或多边形的顶点上。

理解这一步很重要，意味着终于可以不把二次曲线当曲线考虑了，只需考虑交点和顶点。

遍历多边形的每条边，设其两端点分别为 $\mathbf{x}_1, \mathbf{x}_2$ 。

将 $\mathbf{x}_1, \mathbf{x}_2$ 依次代入二次约束方程，可判断点是否在二次约束内。这里分情况讨论：

- 两端点均在二次约束内（含边缘）

此时线段的两端点一定是最终可行域顶点的一部分。

- 其它情况

此时交点数量未知，需要先求出线段与二次约束的交点。

设 $\mathbf{d}_x = \mathbf{x}_2 - \mathbf{x}_1$ ，有线段参数方程：

$$\mathbf{x}(t) = \mathbf{x}_1 + t\mathbf{d}_x, \quad t \in [0, 1]$$

将参数方程代入约束方程：

$$\frac{1}{2}[\mathbf{x}_1 + t\mathbf{d}_x]^\top Q[\mathbf{x}_1 + t\mathbf{d}_x] + \mathbf{p}^\top[\mathbf{x}_1 + t\mathbf{d}_x] + r = 0$$

展开并分离系数，得到标准二次形式：

$$\underbrace{\frac{1}{2}\mathbf{d}_x^\top Q\mathbf{d}_x}_{a}t^2 + \underbrace{\mathbf{x}_1^\top Q\mathbf{d}_x + \mathbf{p}^\top\mathbf{d}_x}_{b}t + \underbrace{\frac{1}{2}\mathbf{x}_1^\top Q\mathbf{x}_1 + \mathbf{p}^\top\mathbf{x}_1 + r}_{c} = 0$$

解得 $t = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ ($t \in [0, 1]$)，代入参数方程即可得到交点的坐标。

交点和在二次约束内（含边缘）的端点一定是最终可行域顶点的一部分。

最后，将所有最终可行域顶点整理为新的多边形。最优解一定是这个多边形的顶点之一，简单遍历即可求解。

5.3.6.2.4. 求解效果

代码编写完成后，使用同样的输入，与杉树求解器的效果进行对比：

COPT solution: (-0.006806, 2.035123)

Elapsed Time: 0.002374s

Our solution: $(-0.006803, 2.035111)$

Elapsed Time: 0.000002s

查看日志可知, 手动编写的解析解求解器, 不仅精度高于通用求解器的迭代解, 且求解速度提升约 10^3 倍。

6. 通讯架构迭代

无下位机的核心问题在于通讯，一个稳定且低延迟的通讯是无下位机控制的基础。

6.1. USB CDC(Communication Device Class, 通信设备类)

在无下位机控制系统的初步尝试中，我们首先尝试采用了 USB CDC 通讯方式。其优势主要在于编写简单，通讯效率高。STM32 HAL 库自带了基本完备的 USB CDC 通讯接口，下位机代码编写时可以直接调用；USB CDC 设备在上位机端也简单地表现为一个串口，上位机代码编写时可调用现成的大量串口通讯库。同时，USB CDC 本质上是封装了的 USB Bulk 通讯，在总线上无其它设备的情况下，可以完全利用 USB FS 的 12Mbit/s 带宽。其通讯延迟也较低。在我们的回环测试中，一次回环通讯（发+收）的平均延迟约为 0.04ms，可以认为基本满足 RoboMaster 机器人的控制需求。

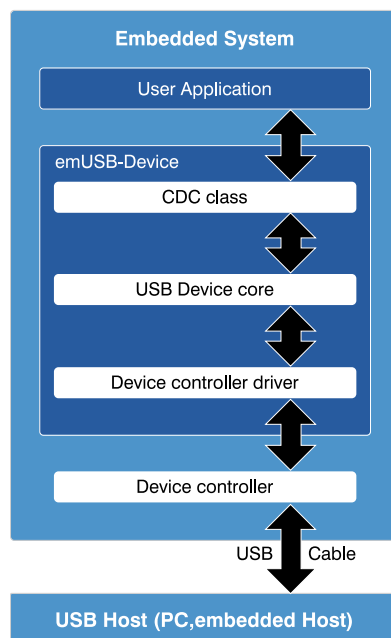


图 8 USB CDC 通讯架构图

以上的种种优点使我们很快地完成了基于 USB CDC 通讯方式的无下位机控制系统，在 2024 年 2 月成功地完整驱动了一辆步兵机器人，并于 2024 年 3 月，让部署了基于 USB CDC 通讯方式的无下位机哨兵参赛 RoboMaster 机甲大师赛（联盟赛）上海站。

但随着研究的深入，我们逐渐意识到 USB CDC 的缺陷：首先，它虽然是基于 USB Bulk 的封装，但它在上位机端表现为一个串口，而串口是典型的字符流。这意味着原本 Bulk 通讯的包信息被遗弃了，上位机代码无法预测 linux CDC 驱动是否会对包进行分割和拼接。于是通讯协议不得不退化回串口通讯常用的复杂形式：头字节+尾校验。上位机需处理校验不通过的各种情况，增加了代码的复杂度。这个复杂度的增加是毫无意义的，USB Bulk 协议中本身就带了由硬件执行的头尾校验，Bulk 协议本身就可以保证包的完整性，但在经过一层封装后，我们反而失去了这个特性，需要浪费额外的带宽来保证通讯的正确。

不仅如此，在我们的测试中，USB CDC 还会无理由的丢弃一些报文，在下位机或上位机以非常高的频率发包时，后发送的报文有很大的概率直接被忽略，且我们无法得到任何关于是否发

送成功的提示。在多次失败的尝试后，我们只能选择在每次发包之间加入数十微秒的延迟来暂时规避这个问题。

进一步地，我们还偶尔遇到一个报文中丢失部分字节的情况，我们在经过研究后推测，由于串口是最古老的通讯设备之一，一些特殊字符组合可能会被当成控制指令而被删除。将串口设置为 raw 模式可以有效解决这个问题，但又带来了一个新的问题：设置串口为 raw 模式的命令可能会出现无缘由的卡顿，有时在等待 30textasciitilde 60 秒后会自行解决，有时可能永远卡死。我们只能设置一个超时等待，在超时后自动重启上位机程序来解决问题。

综上所述，我们认为 USB CDC 不是一个稳定可靠的通讯方式，我们开始着手寻找其它通讯方式。

6.2. EtherCAT（工业以太网）

广东工业大学作为无下位机控制系统的先行者，在通讯方式上经过多次迭代，最终采用了 EtherCAT 技术。

EtherCAT 作为一项高性能、低成本、应用简易、拓扑灵活的工业以太网技术，于 2003 年被引入市场，并于 2007 年成为国际标准。EtherCAT 技术协会（EtherCAT Technology Group, ETG）推广 EtherCAT 技术，并负责对该项技术的持续研发。

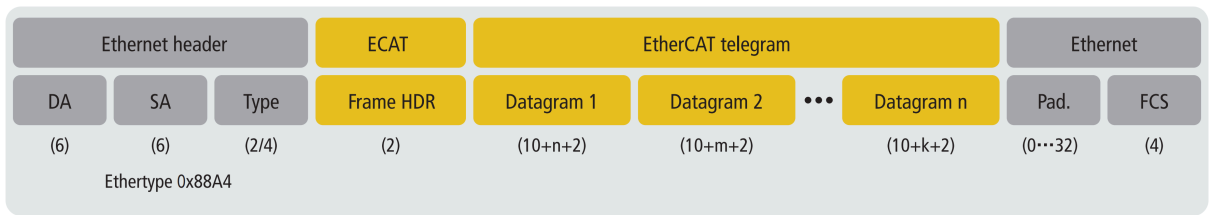


图 9 采用 IEEE 802.3 定义的标准以太网帧传输 EtherCAT 报文

EtherCAT 采用 100 Mbps 全双工以太网（100BASE-TX）作为物理层。因此对于 EtherCAT 主站，在硬件上仅需要一个百兆以太网端口，这一般是 RoboMaster 机器人上的迷你主机的标配。尽管传输速率为 100 Mbps，EtherCAT 从站设备采用从站控制器（ESC）在硬件中高速处理数据帧。因此单个数据帧通过节点的处理延迟非常短，典型延迟为每个节点不到 1 微秒，这使得在非常短的通讯周期内可以传输大量数据。一个包含 1000 个 I/O 节点的网络，其通讯周期可以小至 30 微秒。

在典型的工业场景下，EtherCAT 可以实现低于 100 微秒的总延迟，足以满足绝大多数工业实时控制系统的需求。EtherCAT 网络支持多达 65,535 个设备，而每个设备之间的距离可达 100 米，这使得 EtherCAT 能够覆盖较大范围的工业网络。

如上的优秀特性使我们认为 EtherCAT 是一项理想的技术。因此作为无下位机方案的后来者，我们也对 EtherCAT 技术进行了研究和探索。

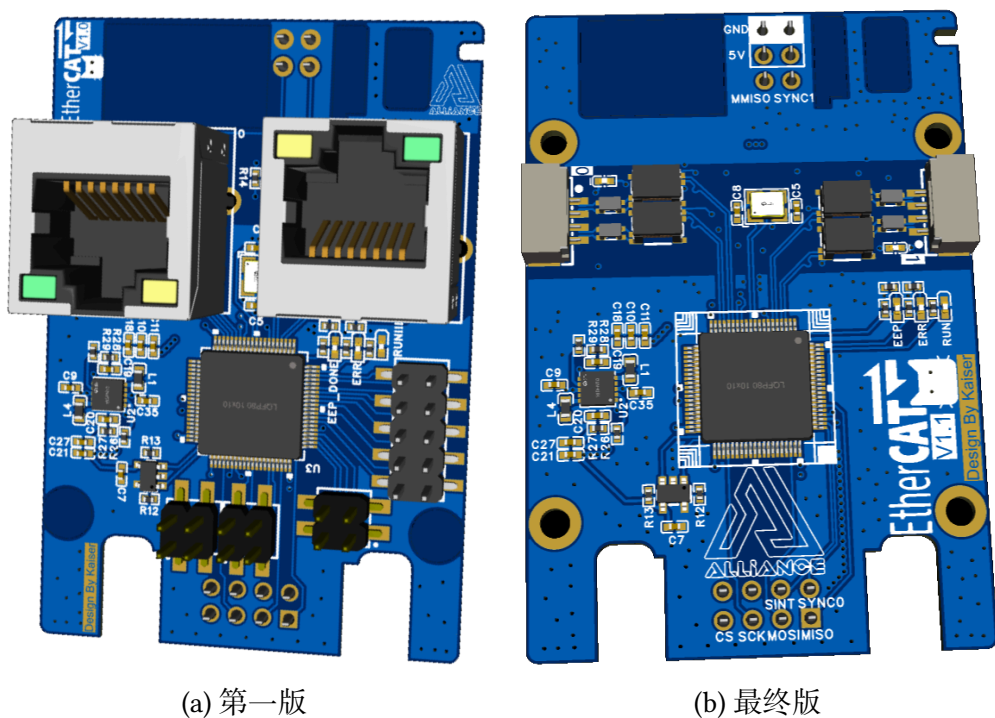


图 10 EtherCAT 从机拓展板

我们选用 ASIX 公司生产的 AX58100 芯片作为我们的从站控制器（ESC），绘制了一块 EtherCAT 从机拓展板。它可以直接插在我们使用的 RoboMaster C 型开发板上，通过 SPI 与 C 板通讯。以求在尽量不改变机器原有接线的情况下，快速部署 EtherCAT。

但在深入研究 EtherCAT 的过程中，我们逐渐意识到它并不像我们一开始所想的那样简单。首先，EtherCAT 的资料在互联网上是缺乏的，EtherCAT 技术协会（ethercat.org）持有大量技术资料，但其中的绝大部分都需要成为他们的会员才能获取。剩余的可访问部分，和互联网上可搜索到的部分，基本上都是浅表的原理讲解和叙述应用层面的优势。这些资料只能让我们对 EtherCAT 从一无所知到一知半解，但很多详细的技术细节，我们可以说是询问无门。至于具体的源码资料，更是难以获取。

在这种情况下，OpenEtherCATsociety (github.com/OpenEtherCATsociety) 作为一个开放且开源的第三方 EtherCAT 实现，为我们指出了一条方向。但同样的，它也没有给出任何关于原理上的解释，我们只能通过看源码的方式对 EtherCAT 进行盲人摸象般的探索。

6.3. USB 同步/等时传输 (Isochronous)

尽管 EtherCAT 方案在纸面上拥有性能和实时性的巨大优势，但其协议栈复杂，遇到问题时查错困难，方案迟迟难以落地，因此我们重新开始了对其它 USB 通讯方案的尝试。

USB 开箱即用，无需额外定制 PCB 的特性依然具有巨大吸引力。原先 USB CDC 通讯方式的问题主要出现在 linux 驱动层上，因此只需要去除驱动层，USB 通讯的稳定性将大大提升。于是使用 libusb 绕过驱动层进行 raw USB 同步传输成为我们的研究方向之一。

同步传输也有“等时传输”的叫法，一般用于要求数据连续、实时且数据量大的场合，如视频设备、音频设备等。其要求严格的定时，数据流按照预定的时间间隔传输，确保稳定的传输速率。这种传输方式主要应用于对延迟要求较高的场景。每次传输周期会分配一个固定的带宽，确保传输信道可以稳定工作，而不会受到总线繁忙程度的影响。这与其他传输模式，如批量传输

(Bulk Transfer) 不同, 后者是根据总线的空闲状态传输数据。在同步传输模式下, 传输的数据不会被错误重传。因为同步传输更看重数据的实时性, 短暂的丢包对于整个应用影响不大 (例如音频或视频流中小部分数据丢失通常不会明显影响体验)。同步传输主要针对实时应用设计, 能够确保数据按照预定的时间顺序进行传输, 从而满足实时处理需求。

若仅看传输特性, 同步传输无疑是四种 USB 传输方式中最适合处理无下位机传输通讯的, 这也是我们首选它的原因。

同时, 由于 USB 相关的资料在互联网上相当多, USB 同步传输方案的研发相对 EtherCAT 通讯方式的研发轻松较多。其难点主要集中在对下位机 HAL 库的修改上。STM32 HAL 库并未提供直接进行同步传输的 Demo, 底层接口也没有较明确的调用文档和使用说明。在多次失败尝试后, 我们将 HAL 库的 Audio Class (音频类, 使用同步传输) 的无关功能删去, 成功同上位机建立了通讯。

成功建立通讯后, 我们马上进行了回环延迟测试, 发现一次回环通讯 (发+收) 的延迟达到了约 4ms, 这显然是不可接受的。仔细研究后, 我们发现对于同步传输的收发, 操作系统使用了一个长度可调的队列来保证传输频率稳定。USB 同步传输在一帧内只收发各一次。虽然队列长度可调, 但最小为 1 的队列长度意味着 OUT 报文不得不滞留一帧。下位机接到报文后, 转发的 IN 报文也很大概率滞留一帧后被发送, 而在我们使用的 USB 2.0 FS 模式下, 一帧意味着 1ms, 消息在帧间的滞留导致了巨大的延迟。要想解决这个问题, 我们除换用更高速的 USB 2.0 HS (其一微帧为 125us, 但需要彻底更改硬件方案) 外, 只有再次更改软件方案。

6.4. USB 批量传输 (Bulk)

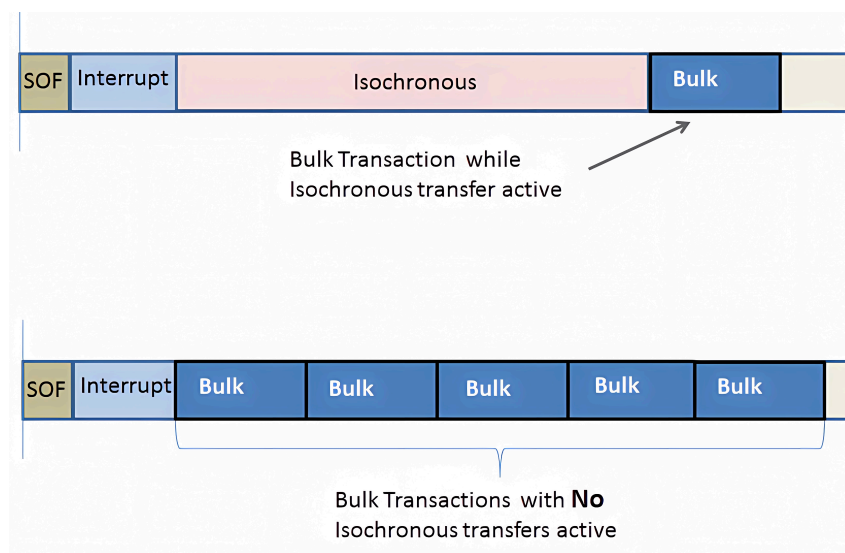


图 11 USB 同步传输与 USB 批量传输的比较

若想在继续使用 USB 2.0 FS 的同时, 规避同步传输消息在帧间滞留的问题, USB 批量传输显然是唯一的选择。

USB 批量传输是 USB CDC 通讯方式的底层介质, 其并不为实时传输而设计。一开始因其最低的传输优先级, 硬件级别的强制性丢包重传, 和较短的允许报文长度 ($l = 64$ bytes) 并不被我们作为首选考虑。

但已知的测试结果表明 USB CDC 通讯方式是 USB 2.0 FS 连接方式下平均回环延迟最低（约 0.04ms），传输带宽最大（总线空闲时可达 12MB/s）的。

在低延迟的要求下，USB Bulk 较短的允许报文长度反而成了优势。其在一个 USB 帧内可在任意时间完成多次不同向的传输，因此消息不会在帧间滞留。同时，C 板与迷你主机之间通常使用 USB 线直连，即使需要使用 USB Hub（拓展坞），也可以简单地购买 USB 2.0 HS 或 USB 3.0 的 Hub，利用 USB Hub 的协议提升特性，让总线重新拥有大量空闲时间片，保证优先级最低的 USB Bulk 也能稳定传输。

尝试了多种方案后，USB Bulk 方案被作为最终方案。虽然前期尝试的三种方案均未被采用，但项目的快速平稳落地离不开前期研究积累的大量经验。

Index of Figures

图 1	Foxglove 远程可视化界面	7
图 2	二维和三维坐标的轴方向约定（与 ROS 的约定相同）	16
图 3	RMCS 启动流程	17
图 4	在平面内考虑云台限位	20
图 5	在三维空间（YawLink 系）中考虑云台限位	20
图 6	在 YawLink 坐标系下计算控制误差	21
图 7	可行域的形状仅与 α 相关	31
图 8	USB CDC 通讯架构图	39
图 9	采用 IEEE 802.3 定义的标准以太网帧传输 EtherCAT 报文	40
图 10	EtherCAT 从机拓展板	41
图 11	USB 同步传输与 USB 批量传输的比较	42